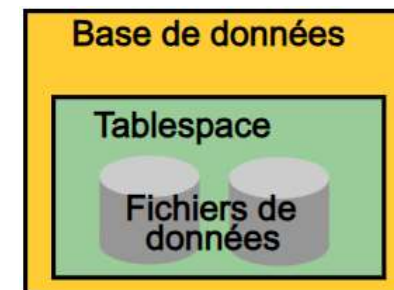
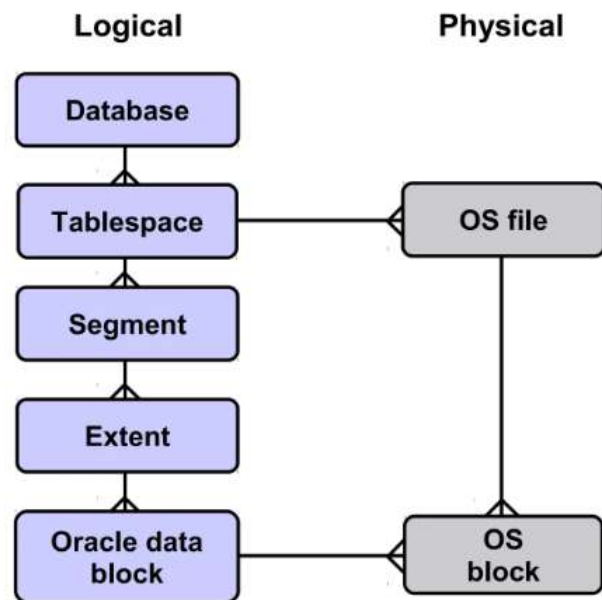


Chapitre 07 : Gestion des tablespaces et des fichiers de données DB Oracle

Storage Structures



Objectifs du chapitre

Objectifs

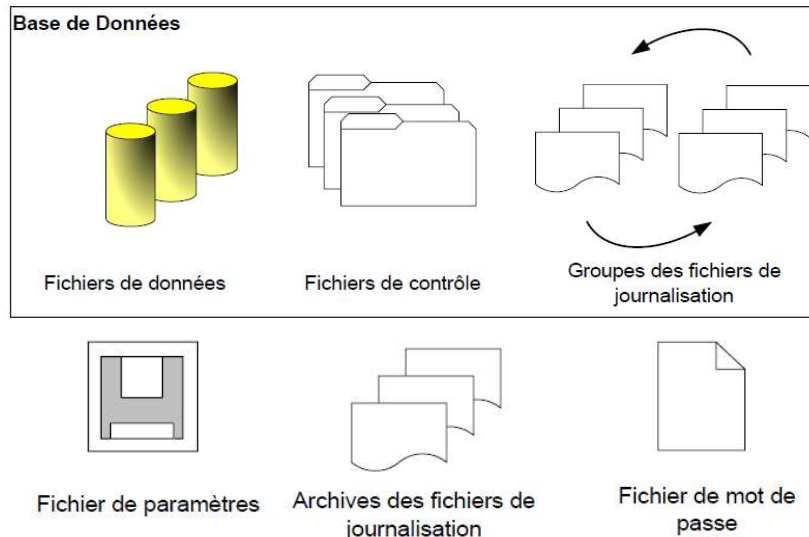
A la fin de ce chapitre, vous pourrez :

- **définir la fonction des tablespaces et des fichiers de données**
- **créer des tablespaces**
- **gérer des tablespaces**
- **créer et gérer des tablespaces à l'aide d'Oracle-Managed Files (OMF)**
- **Obtenir des informations sur les tablespaces et le fichier des données**

Structures physiques et logiques d'une BD Oracle

Structure physique :

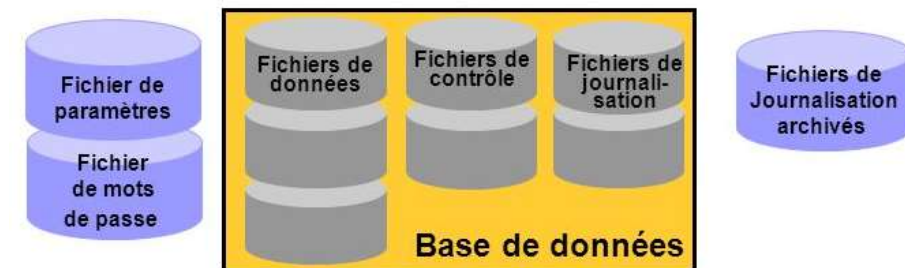
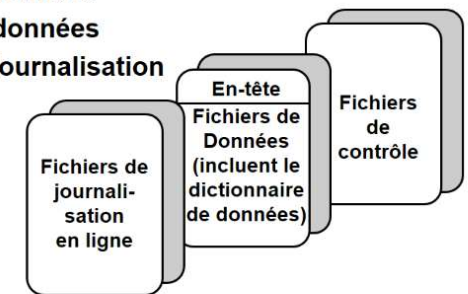
- La base de données est constituée d'un ensemble de **fichiers physiques** situés sur les disques durs du serveur hébergeant la base.
- La structure physique d'une base de données Oracle comprend trois types de fichier : **fichiers de contrôle**, **fichiers de données** et **fichiers de journalisation**.
- La Figure ci-dessous montre les différents types de ces fichiers, il est à noter que les **archives des fichiers de journalisation (fichiers de journalisation archivés)**, le **fichier de paramètres** ainsi que le **fichier des mots de passe** ne font pas partie prenante de la base de données, mais y sont étroitement liés.



Structure physique

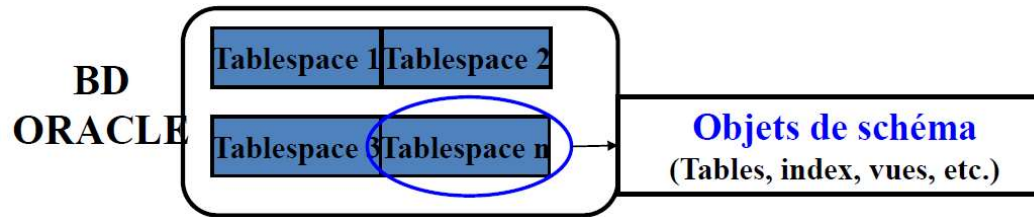
La structure physique comprend trois types de fichier :

- Fichiers de contrôle
- Fichiers de données
- Fichiers de journalisation



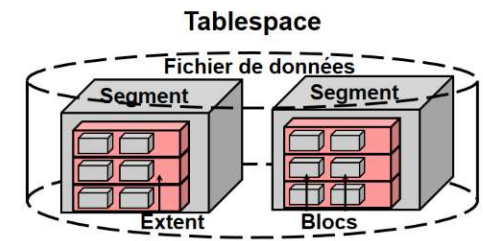
Structures physiques et logiques d'une BD Oracle

Structure logique :

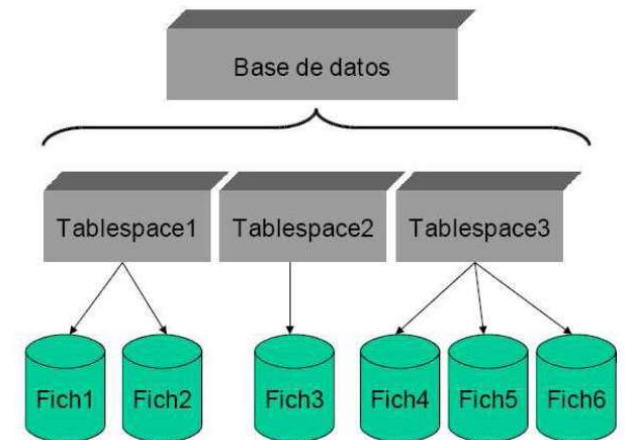


Structure logique

- La structure logique définit le mode d'utilisation de l'espace physique d'une base de données.
- Cette structure possède une hiérarchie composée de tablespaces, de segments, d'extents et de blocs.



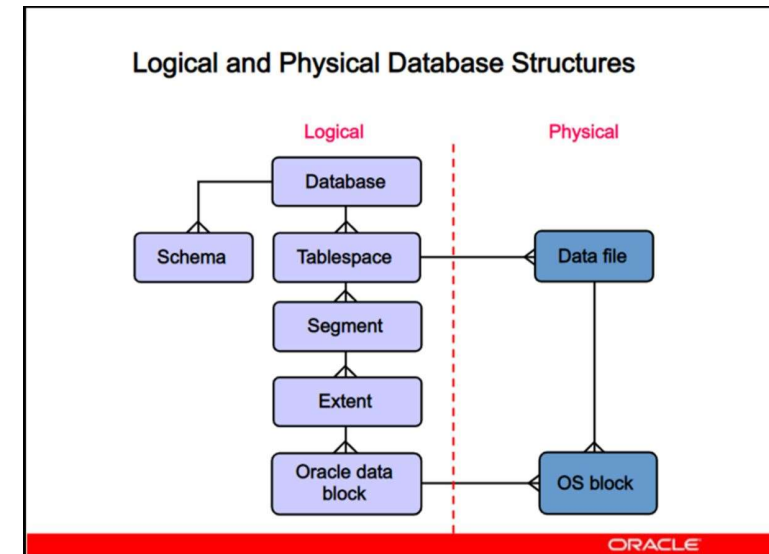
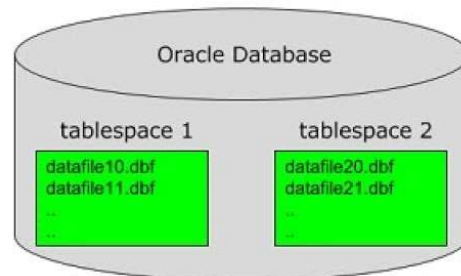
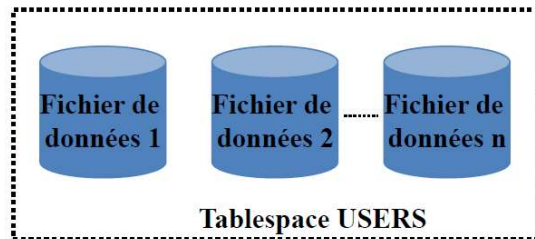
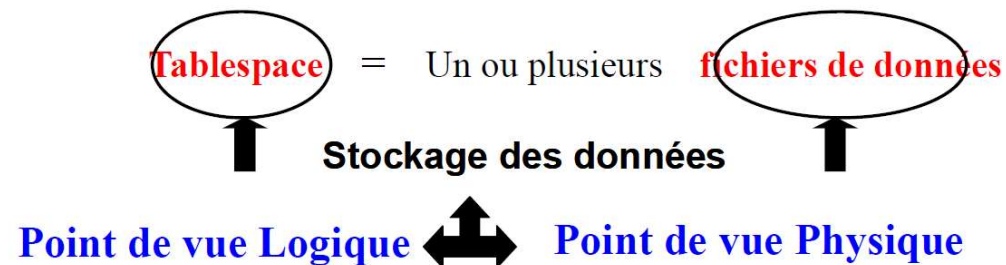
ORACLE



- La hiérarchie de la structure logique se présente comme suit :
 - Une **base de données** Oracle contient au moins un **tablespace**.
 - Un **tablespace** comporte un ou plusieurs **segments**.
 - Un **segment** est composé d'**extents**.
 - Un **extent** est composé de **blocs de données contigus**.
 - Un **bloc** représente la plus **petite unité** sur laquelle portent les opérations de lecture et d'écriture.
- L'**architecture** d'une **base de données Oracle** contient les **structures logiques** et **physiques** qui constituent la base de données.
 - La **structure physique** comprend les fichiers de contrôle, les fichiers de journalisation en ligne (online redo log) et les fichiers de données qui constituent la base de données.
 - La **structure logique** est composée de tablespaces, de segments, d'extents (ensemble de blocs contigus) et de blocs de données.
- Le serveur Oracle permet un **contrôle précis de l'utilisation de l'espace disque** à l'aide de **tablespaces** et de structures de stockage logiques constituées de segments, d'extents et de blocs de données.

Structures physiques et logiques d'une BD Oracle

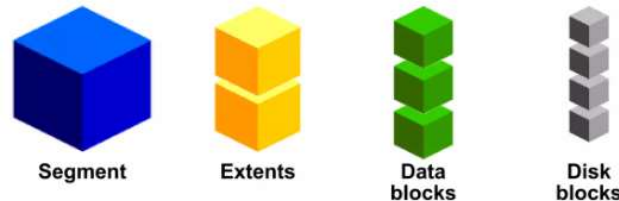
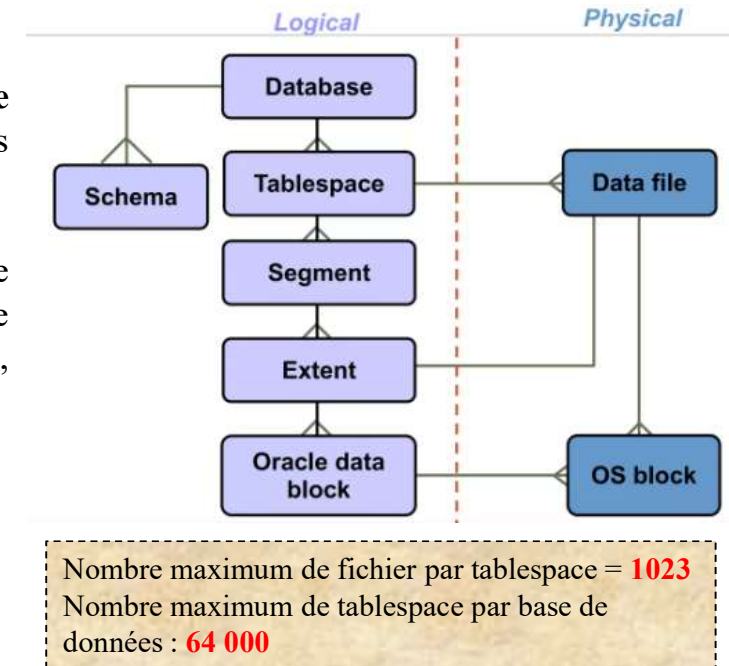
Correspondance : Structure Logique et Structure Physique d'une BD:



Structures physiques et logiques d'une BD Oracle

Gestion du stockage d'une BD Oracle :

- La **séparation** des **structures logiques** et **physiques** d'une base de données facilite le **contrôle poussé** de la **gestion de l'espace disque**. L'administrateur peut configurer les paramètres d'allocation d'espace aux composants physiques et logiques de la base de données.
- Pour utiliser efficacement l'espace du disque dur, il est important de connaître les relations entre les composants physiques et logiques de la base de données. Il est important également de savoir comment l'espace est alloué dans la base de données. Comme on l'a vu précédemment, la base de données est divisée en **zones d'espace logique** plus petites, appelées **tablespaces**.



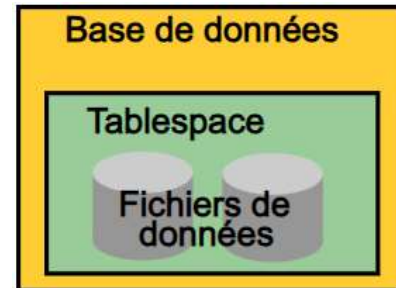
- ✓ Un **tablespace** est constitué **d'un ou plusieurs** fichiers de données (**datafiles**) ; par contre un fichier de données ne peut appartenir qu'à un seul tablespace. Un **fichier de données** est créé **automatiquement** par le serveur Oracle chaque fois que vous **créez le tablespace** ou que vous avez **ajouté un nouveau fichier** au tablespace. La quantité de disque occupé par le fichier de données est spécifiée par l'administrateur de la base de données. Chaque fois que le fichier est créé, **l'espace est automatiquement réservé** sur disque.
- ✓ Après la création d'un tablespace, vous pouvez ajouter d'autres fichiers de données. Un **fichier de données** peut être **modifié** par l'administrateur de la base de données **après sa création**.
- ✓ Dès lors qu'un **fichier de données est créé** pour un tablespace, il est **attaché** à ce tablespace, et il va pouvoir être **détaché** uniquement par la **destruction** du tablespace.

Tablespaces – Fichiers de données

Tablespaces :

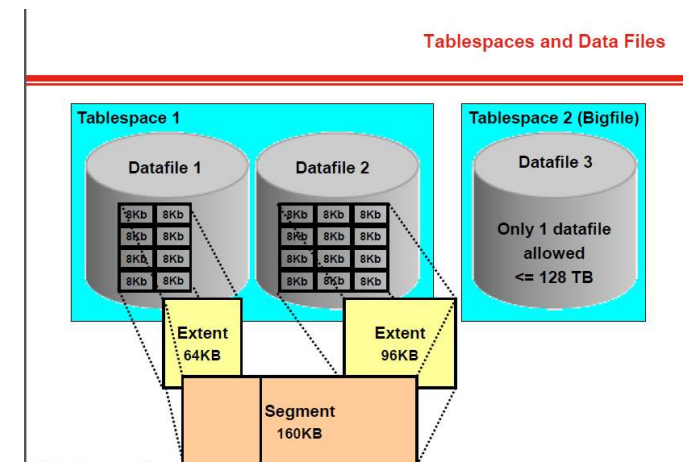
Les **données** d'une base de données Oracle sont stockées dans des **tablespaces**.

- Une base de données Oracle peut être divisée en plus petites zones logiques d'espace appelées **tablespaces**.
- Un tablespace ne peut appartenir qu'à une seule base de données à la fois.
- Chaque tablespace est constitué d'un ou de plusieurs fichiers du système d'exploitation, appelés **fichiers de données**.
Un fichier de données **appartient** à un et un **seul** tablespace.
- Un **tablespace** peut **s'étendre** soit par **ajout** (on-line) d'un fichier, soit par **auto-extension** du fichier du tablespace.
- Les tablespaces peuvent être mis **hors/en ligne** lorsque la base de données est active.
- A l'exception du tablespace **SYSTEM** ou des tablespaces contenant un **segment d'annulation actif**, les tablespaces peuvent être mis hors ligne sans interruption de la base de données.
- Les tablespaces peuvent être accessibles en **lecture et écriture** ou en **lecture seule**.



Fichiers de données (structure physique, non logique) :

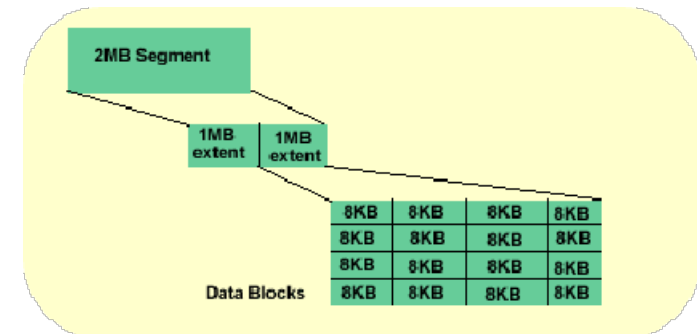
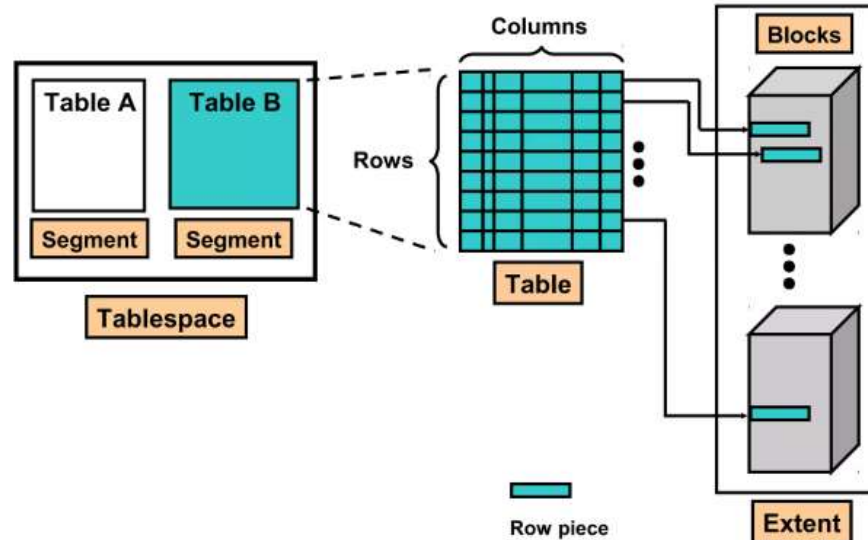
- Chaque **tablespace** d'une base de données Oracle contient un ou plusieurs fichiers appelés **fichiers de données**. Ces fichiers sont des **structures physiques** conformes au **système d'exploitation** sur lequel s'exécute le serveur Oracle.
- Un fichier de données ne peut appartenir qu'à un **seul tablespace**.
- Un serveur Oracle crée un fichier de données dans un tablespace en allouant **l'espace disque défini** et un **petit espace supplémentaire**.
- L'administrateur de base de données peut **modifier la taille** des fichiers de données après leur création ou indiquer que leur taille peut **augmenter de façon dynamique** à mesure que celle des objets du tablespace s'accroît.



Tablespaces - Segments

Segments :

- Un **tablespace** est constitué de **segments**. Un tablespace peut contenir **un ou plusieurs segments**.
- Un segment est l'**espace alloué** pour un **type spécifique de structure de stockage logique** dans un tablespace.
- Les **segments d'index**, **segments temporaires**, **undo segments** et **segments de données** représentent quelques exemples de segments.
- Un segment, tel qu'un segment de données, peut être **réparti** sur **plusieurs fichiers** appartenant au **même tablespace**.
- Le niveau suivant de la structure logique d'une base de données est l'**extent**. Un extent est un **ensemble de blocs contigus**. Chaque segment est constitué d'un ou plusieurs extents. Un extent **ne peut pas être stocké** sur plusieurs fichiers de données.



Tablespaces - Segments



Principaux types de Segments

1) Segments de données (data)

- Servent à stocker les données contenues dans :
 - Les tables des utilisateurs.
 - Les tables du dictionnaire de données.

2) Segments d'index

- Servent à stocker les informations d'indexations relatives aux tables.

3) Segments d'annulation (rollback)

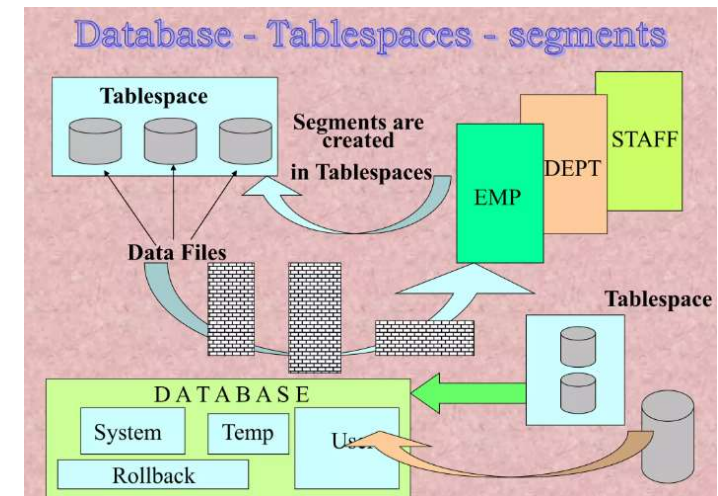
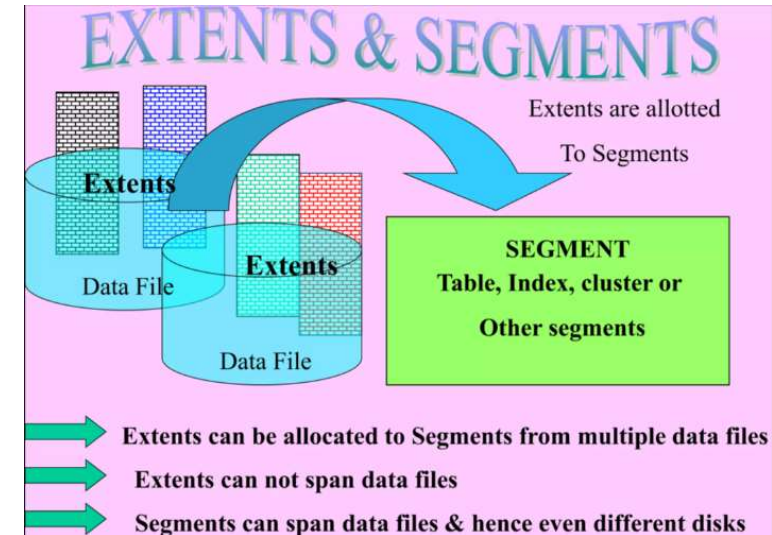
- Sont utilisés pour la gestion des transactions.
- Contiennent les copies des données avant modification permettant ainsi d'annuler la transaction en cas de besoin.

4) Segments temporaires

- Sont utilisés pour allouer de l'espace du travail afin d'évaluer certaines requêtes (opérations de tri).
- Nettoyer par SMON.

5) Segment de démarrage (bootstrap)

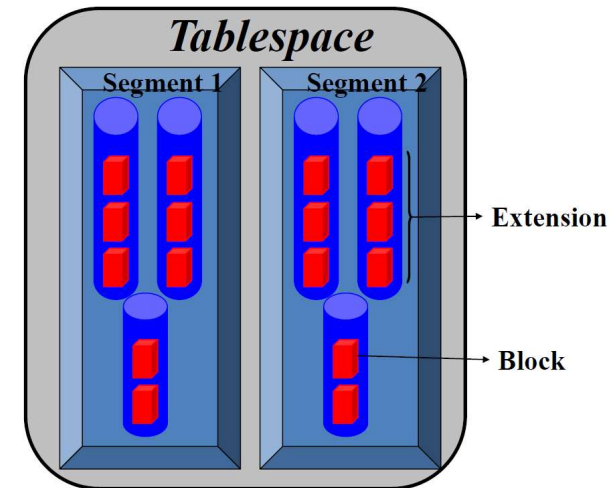
- Utilisé une seule fois lors de la création de la base de données.



Tablespaces – Extents & Data Block

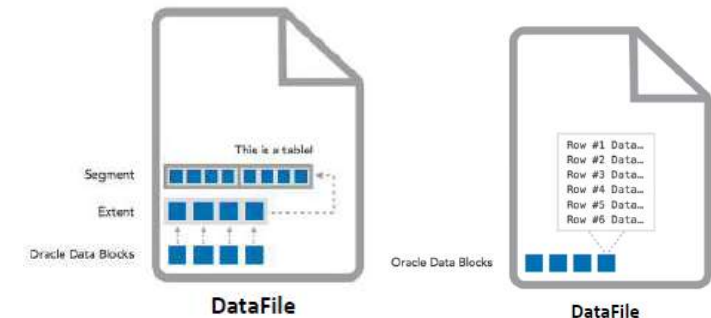
Extents (extensions) :

- Les **extents** permettent d'**allouer de l'espace à un segment**.
- Un segment peut être constitué d'**un ou de plusieurs extents**.
 - Lorsque vous créez un segment, celui-ci contient au moins un extent.
 - A mesure que la taille du segment augmente, des extents lui sont ajoutés.
 - L'administrateur de base de données peut ajouter manuellement des extents à un segment.
- Un **extent** est constitué de **blocs Oracle contigus**.
- Un extent **ne peut pas s'étendre** sur plusieurs fichiers de données. Il doit donc appartenir à un **seul fichier** de ce type.



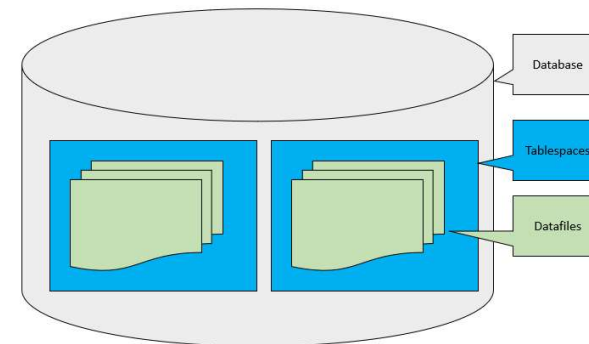
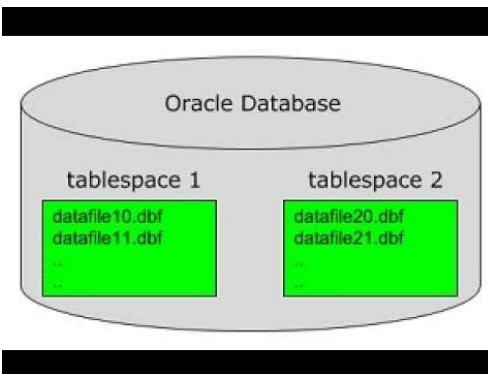
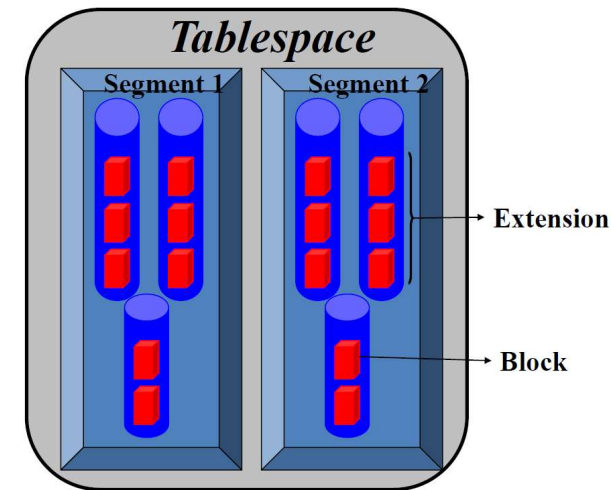
Blocs de données (Data Block) :

- Les données d'une base Oracle sont stockées dans les blocs de données.
- Un **bloc de données** correspond à un **nombre d'octets** spécifique d'**espace physique** sur le disque.
 - La **taille** des blocs est définie lors de la **création** de chaque **tablespace**.
- Taille peut être : 2, 4, 8, 16, 32 Ko (par **défaut 8 ko**)
- La base de données utilise et alloue l'espace disponible dans les blocs de données Oracle.



Utilisation des Tablespaces

- ❑ Contrôle de l'allocation d'espace et affectation de quotas aux utilisateurs.
- ❑ Contrôle de disponibilité des données par la mise **online** ou **offline** des tablespaces.
- ❑ Distribution du stockage des données sur plusieurs dispositifs
 - pour améliorer les E/S;
 - pour **réduire la contention** sur un seul disque.
- ❑ Exécution de **sauvegarde partielle**.
- ❑ Conservation de volumes importants de **données statiques** sur des dispositifs en **lecture seule**.





Les types de tablespaces

L'administrateur de base de données crée des tablespaces pour **améliorer le contrôle** et **faciliter la gestion** de la base. Le serveur Oracle accepte deux types de tablespace : le tablespace **SYSTEM** et tous les autres tablespaces.

▪ Le tablespace System :

- Indispensable au fonctionnement de la base de donnée.
- Il contient les informations du **dictionnaire de données**, les définitions des **procédures stockées**, des **packages**
- Ne devrait pas contenir de données utilisateurs.

▪ Les tablespaces non system :

- Permettent **plus de flexibilité** dans l'administration de la BD.
- Composées de **segments de données**, d'**indexes**, de **rollback** et de **segments temporaires**.

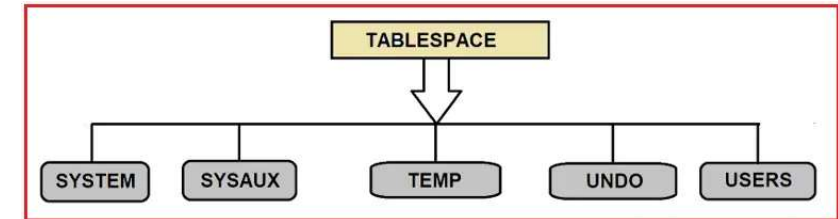
Types de tablespace

- **Le tablespace SYSTEM :**
 - est créé en même temps que la base de données,
 - contient le dictionnaire de données,
- **Le tablespace non SYSTEM**
 - Les tablespaces permanentes (BigFile et SmallFile)
 - Les tablespaces temporaires
 - Les tablespaces d'annulations

Les types de tablespaces

Une base de données **Oracle** possède au minimum **04 tablespaces** :

- **Tablespace SYSTEM** ; dictionnaire de données.
- **Tablespace SYSAUX** ; system auxiliaire composants oracle.
- **Tablespace UNDO** ; segments d'annulation.
- **Tablespace TEMPORAIRE** ; segments temporaires.
- Puis des **tablespaces applicatifs** spécifiquement créés pour des **données applicatives** et **utilisateurs**.

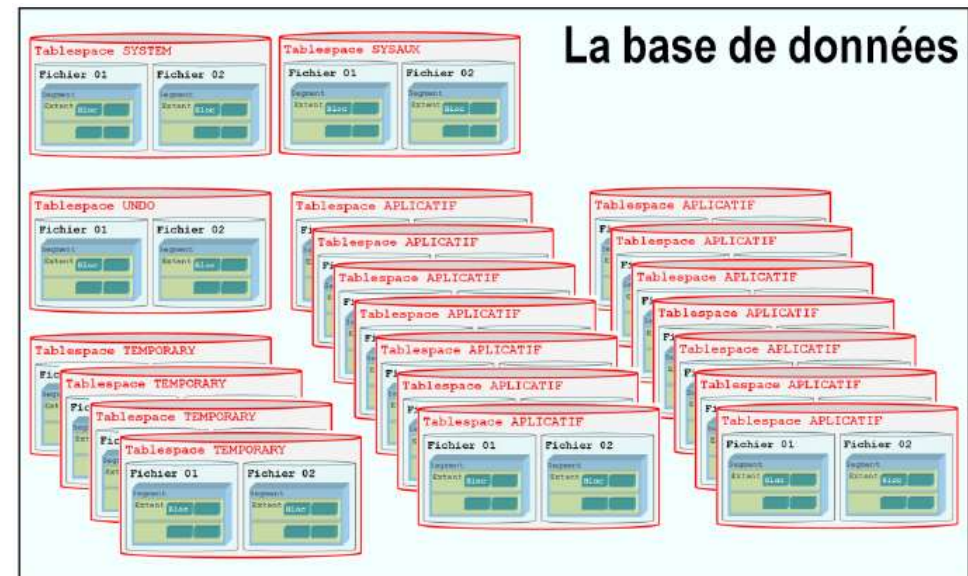


Mais pourquoi autant de tablespaces ?

Simplement pour **bien ranger** les objets par type (application, index, tables, etc...) et ainsi permettre une **administration plus fluide** et plus **souple**, une meilleure **optimisation**, et **performances**.

1) Tablespace SYSTEM :

- Il s'agit de l'emplacement où Oracle garde toute l'information du **catalogue** exigée pour le fonctionnement de la base de données.
- **SYSTEM** est utilisé pour les fonctionnalités principales de la base (ex : les tables du **dictionnaire des données**).

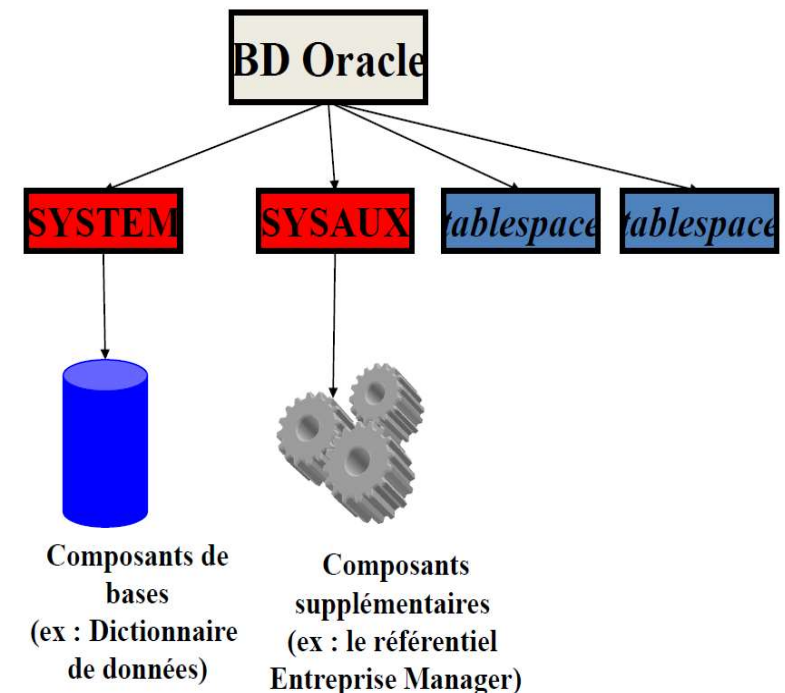


Les types de tablespaces

2) Tablespace SYSAUX :

- Un deuxième tablespace « **SYSAUX** » est **créé automatiquement** à la **création** de la base de données. Il contient les **objets système complémentaires** qui permettent ainsi de diminuer le temps d'attente pour les **lectures des informations système**.
- Les composants d'Oracle supplémentaires (ex : le **référentiel Entreprise Manager**) l'utilisent comme **tablespace par défaut** afin de **réduire** la charge à **SYSTEM**.
- Il possède les mêmes attributs de sécurité que le tablespace **SYSTEM**.
- Les deux tablespaces « **SYSTEM** » et « **SYSAUX** » sont utilisés uniquement par Oracle ; il faut prendre soin de **ne pas stocker des objets utilisateur** dans ces **deux tablespaces**.

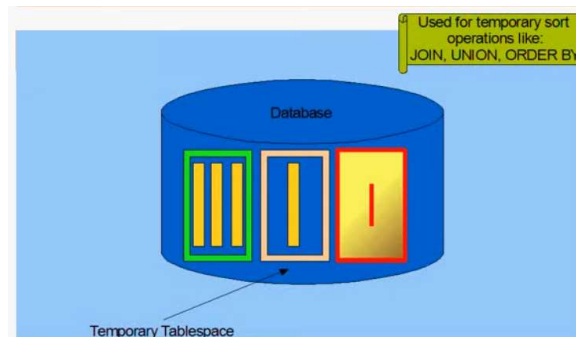
BD Oracle & Tablespace



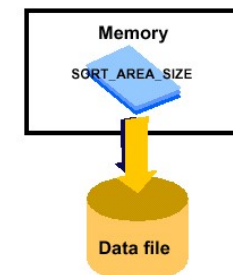
Les types de tablespaces

3) Le tablespace temporaire :

- Lors d'importantes **opérations de tri** (telles que select distinct, union et create index), si la taille de la **zone PGA** du processus utilisateur ne **suffit pas**, Oracle va **stocker dans les tablespaces** de la base de données des informations concernant le tri des enregistrements avant de retourner l'information aux utilisateurs.
- En raison de leur **nature dynamique**, ces espaces de tri ne devraient pas être stockés avec d'autres types de segments. Lorsqu'un tablespace temporaire « **TEMPORARY** » est défini, un **segment de tri** est aussi créé. Celui-ci est **capable de croître** si nécessaire afin de pouvoir héberger toutes les opérations de tri de données, et existe jusqu'à ce que la **base de données soit fermée puis redémarrée**.
- Lors de la création de la base de données, vous avez la possibilité de définir un tablespace avec la clause « **DEFAULT TEMPORARY TABLESPACE** » de la commande « **CREATE DATABASE** ». Certains utilisateurs d'une base de données Oracle peuvent avoir besoin de volumes de stockage temporaires beaucoup plus grands que ceux de tous les autres utilisateurs de l'application.
- Dans ce cas, vous pouvez créer **plusieurs tablespaces temporaires** « **TEMPORARY** », pour distribuer les espaces de stockages des utilisateurs ayant des **besoins semblables** sur les **mêmes tablespaces**.



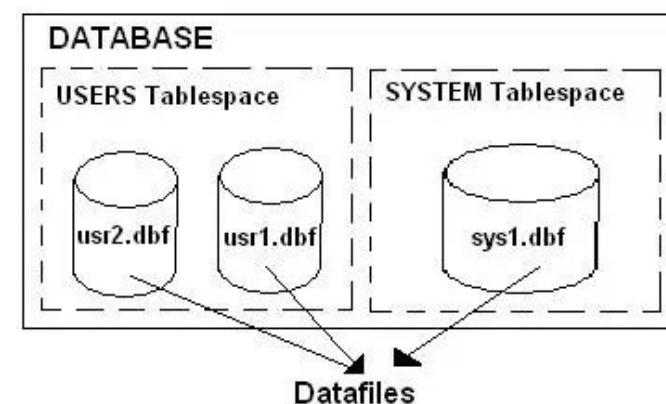
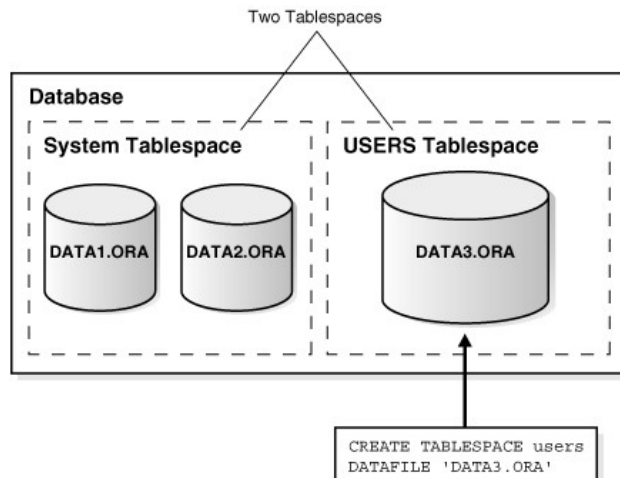
Temporary Segment



Les types de tablespaces

4) Le tablespace permanent :

- Ce **tablespace** est un espace **logique** qui contient les objets stockés dans la BD comme les **tables** ou les **indexes**. Il est composé d'au moins un **datafile** qui est **physiquement** présent sur le serveur de BD (Chaque **BD** possède au moins un tablespace appelé **SYSTEM**).
- On appelle **Tablespaces permanents**, les tablespaces autres que **TBS UNDO** et **TBS TEMPORARY**.
- Il est également possible lorsque vous créez la base de données de spécifier le **tablespace permanent par défaut** avec la clause « **DEFAULT TABLESPACE** » de la commande « **CREATE DATABASE** ».
- Tous les objets qui ne comportent pas, dans la syntaxe de création, les mentions de stockage dans un tablespace, sont **stockés automatiquement** dans le **tablespace permanent par défaut**.



Les types de tablespaces

5) Le tablespace UNDO :

- Le tablespace **UNDO** comme son nom l'indique, est réservé exclusivement à l'**annulation** des **commandes DML** (UPDATE, INSERT, ...). Ce tablespace est **unique à une instance**, donc il y a autant d'UNDO que d'instance de base de données (System Identifier - **SID**).
- Lorsqu'on exécute l'ordre **DELETE** par exemple, Oracle commence par copier les lignes à supprimer dans le tablespace **UNDO**. Ensuite, Oracle indique que les blocs contenant les données dans le tablespace d'origine sont libres. Un **ROLLBACK** permet de revenir en arrière alors que le **COMMIT** supprimera les lignes du tablespace **UNDO**.
- Lorsque vous créez un tablespace « **UNDO** », Oracle gère le stockage, la rétention et l'emploi de l'espace pour les données de rollback par l'intermédiaire de la fonction **SMU (System-Managed Undo)**. Aucun **objet permanent** n'est placé dans le tablespace **UNDO**.

6) Le tablespace applicatif :

- Un **tablespace applicatif** typique contient tous les **objets** principaux **associés à une application**. L'important **volume de lectures-écritures** dont elles font l'objet, justifie l'isolation de ces tables dans leur propre tablespace, séparant ainsi leurs fichiers de données des autres fichiers de données dans la base. La **répartition** de ces fichiers sur des disques différents peut de plus améliorer les performances (grâce à une **réduction de la contention** lors des accès au disque) et **simplifier leur gestion**.
- Les index ne devraient pas être stockés dans le même tablespace que les tables de données sur lesquelles ils ont été définis**, car ces deux types de structures font l'objet de **nombreuses opérations de lectures-écritures concurrentes** lors des manipulations de données et des interrogations.

Les types de tablespaces

Rappelez-vous, la syntaxe SQL de création de la base de données comporte d'abord la création d'un tablespace « **SYSTEM** » et d'un tablespace « **SYSAUX** », ainsi que la création de deux tablespaces « **TEMP** » et « **UNDO** ».

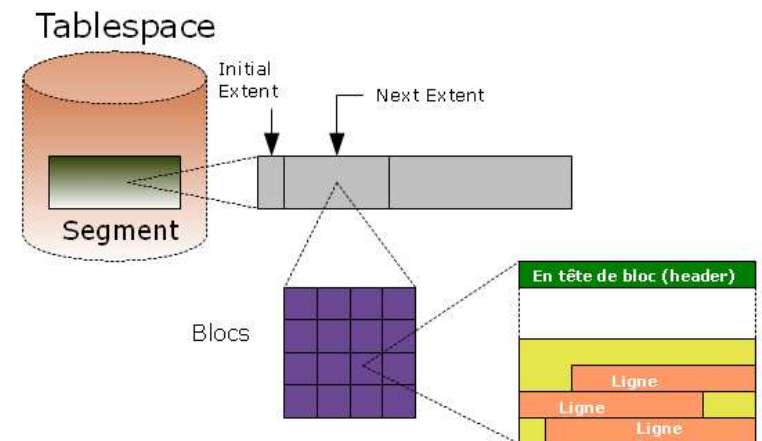
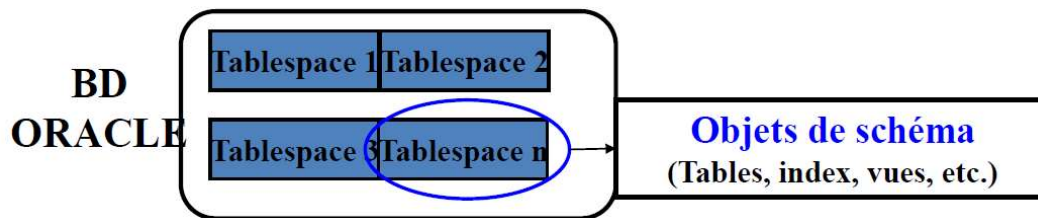
```
SQL> CREATE DATABASE "ambre"
 2  MAXINSTANCES 8 MAXLOGHISTORY 1 MAXLOGFILES 16
 3  MAXLOGMEMBERS 3 MAXDATAFILES 1024
 4  DATAFILE SIZE 700M AUTOEXTEND ON NEXT 10240K
 5      MAXSIZE UNLIMITED EXTENT MANAGEMENT LOCAL
 6  SYSAUX DATAFILE SIZE 550M AUTOEXTEND ON NEXT 10240K
 7      MAXSIZE UNLIMITED
 8  DEFAULT TEMPORARY TABLESPACE TEMP TEMPFILE SIZE 20M
 9      AUTOEXTEND ON NEXT 640K MAXSIZE UNLIMITED
10  UNDO TABLESPACE "UNDOTBS1" DATAFILE SIZE 200M
11      AUTOEXTEND ON NEXT 5120K MAXSIZE UNLIMITED
12  DEFAULT TABLESPACE "USERS" LOGGING DATAFILE SIZE 5M
13      AUTOEXTEND ON NEXT 5M MAXSIZE UNLIMITED
14  CHARACTER SET WE8MSWIN1252 NATIONAL CHARACTER SET AL16UTF16
15  LOGFILE GROUP 1 SIZE 1G, GROUP 2 SIZE 1G, GROUP 3 SIZE 1G,
16      GROUP 4 SIZE 1G, GROUP 5 SIZE 1G, GROUP 6 SIZE 1G
17  USER SYS IDENTIFIED BY "&&sysPassword"
18  USER SYSTEM IDENTIFIED BY "&&systemPassword";
```

```
SQL> CREATE DATABASE "tpdba"
 2      MAXINSTANCES 8
 3      MAXLOGHISTORY 1
 4      MAXLOGFILES 24
 5      MAXLOGMEMBERS 2
 6      MAXDATAFILES 1024
 7  DATAFILE SIZE 300M AUTOEXTEND ON
 8      NEXT 10240K
 9      MAXSIZE UNLIMITED
10      EXTENT MANAGEMENT LOCAL
11  SYSAUX DATAFILE SIZE 120M AUTOEXTEND ON
12      NEXT 10240K
13      MAXSIZE UNLIMITED
14  DEFAULT TEMPORARY TABLESPACE TEMP TEMPFILE SIZE 20M
15      AUTOEXTEND ON NEXT 640K MAXSIZE UNLIMITED
16  UNDO TABLESPACE "UNDOTBS1" DATAFILE SIZE 200M
17      AUTOEXTEND ON NEXT 5120K MAXSIZE UNLIMITED
18  CHARACTER SET WE8MSWIN1252
19  NATIONAL CHARACTER SET AL16UTF16
20  LOGFILE GROUP 1 SIZE 10240K,
21      GROUP 2 SIZE 10240K,
22      GROUP 3 SIZE 10240K
23  USER SYS IDENTIFIED BY "&&sysPassword"
24  USER SYSTEM IDENTIFIED BY "&&systemPassword";
```


Gestion de l'espace dans les tablespaces

Gestion de l'allocation d'espace

- Lors de la création d'un **segment** pour un **objet** (table, index...), Oracle crée un **extent** dans le tablespace cible de l'objet.
- Lorsqu'on remplit ce **segment**, Oracle remplit les **blocs de données** qui constituent l'extent jusqu'à remplir l'extent entièrement et crée un **nouvel extent** si le précédent est plein.



Il existe 2 modes de gestion :

1. Soit **localement**, les informations **extents** libres et alloués sont stockées dans l'entête des fichiers de données du **tablespace**.
2. Soit par le **dictionnaire de données**, les informations extents libres et alloués sont stockées dans les tables du **dictionnaire de données** (Tablespace SYSTEM).

Gestion de l'espace dans les tablespaces



Gestion de l'espace dans les tablespaces

- Tablespace géré localement :
 - Extents libres gérés dans le tablespace
 - Un bitmap est utilisé pour enregistrer des extents libres
 - Chaque bit correspond à un bloc ou à un groupe de blocs
 - La valeur des bits indique si ceux-ci sont disponibles ou utilisés
- Tablespace géré au moyen du dictionnaire :
 - Les extents libres sont gérés par le dictionnaire de données.
 - Les tables appropriées sont mises à jour lorsque les extents sont alloués ou libérés.

ORACLE

Tablespaces gérés localement

- La contention au niveau des tables du dictionnaire de données est réduite.
- Aucune annulation n'est générée lors de l'allocation ou de la libération d'espace.
- Aucune fusion n'est requise.

```
CREATE TABLESPACE userdata
DATAFILE '/u01/oradata/userdata01.dbf' SIZE 500M
EXTENT MANAGEMENT LOCAL UNIFORM SIZE 128K;
```

Tablespaces gérés au moyen du dictionnaire

- Les extents sont gérés dans le dictionnaire de données
- Chaque segment stocké dans le tablespace peut posséder une clause de stockage différente
- Une fusion est requise

```
CREATE TABLESPACE userdata
DATAFILE '/u01/oradata/userdata01.dbf'
SIZE 500M EXTENT MANAGEMENT DICTIONARY
DEFAULT STORAGE
(initial 1M NEXT 1M PCTINCREASE 0);
```

Gestion de l'espace dans les tablespaces

Exemple 1 (tablespace g  r   par le dictionnaire de donn  e)

Create tablespace data

Datafile

```
'd:\oracle\oradata\iges5\data01.dbf' size 30M,  
'e:\oracle\oradata\iges5\data02.dbf' size 20M  
autoextend on next 5M maxsize 60M
```

Extent management dictionary

Minimum extent 150K

Default storage (initial 300K

next 300K

pctincrease 0

minextents 1

maxextents 50)

Online;

Exemple 2 (tablespace g  r   localement avec des extents uniform)

Create tablespace indx

Datafile

```
'd:\oracle\oradata\iges5\indx01.dbf' size 100M,  
Extent management local uniform size 128k;
```

[EXTENT MANAGEMENT

DICTIONARY

| LOCAL [AUTOALLOCATE | UNIFORM [SIZE valeur [K | M]]]]

- MANAGEMENT DICTIONARY: le tablespace est g  r   avec le dictionnaire de donn  es.
- MANAGEMENT LOCAL: le tablespace est g  r   localement et une partie de son espace est r  serv  e pour sa gestion.
 - A la diff  rence des tablespaces standards g  r   au niveau du dictionnaire de donn  es, la gestion de l'espace physique (allocation / lib  ration de blocs) se fait dans l'ent  te du fichier(s) du tablespace. Une table binaire d'allocation (bitmap) y est maintenue.
 - Avantages:
 - pas de contention en mise    jour au niveau du dictionnaire
 - ...
- AUTOALLOCATE: la taille des extents est g  r  e par oracle, c'est l'option par d  faut
- UNIFORM: il est d  fini une taille uniforme des extents en k octets ou en M octets . La taille par d  faut est de 1 m  ga Octet

La création d'un tablespace : Syntaxe abrégé (qq options)



```
CREATE TABLESPACE nom_tablespace
DATAFILE 'chemin d'accès du fichier' SIZE
valeur [K|M]
[MINIMUM EXTENT valeur [K|M] ]
[MAXIMUM EXTENT valeur [K|M] ]
[DEFAULT STORAGE
( [INITIAL valeur [K|M]]
[NEXT valeur [K|M]]
[MINEXTENTS nombre]
[MAXEXTENTS nombre]
[PCTINCREASE pourcentage) ) ;
```



```
CREATE TABLESPACE nom_tablespace
DATAFILE chemin d'accès du fichier' SIZE valeur [K|M]
[MINIMUM EXTENT valeur [K|M] ]
[MAXIMUM EXTENT valeur [K|M] ]
[DEFAULT STORAGE
( [INITIAL valeur [K|M]]
[NEXT valeur [K|M]]
[MINEXTENTS nombre]
[MAXEXTENTS nombre]
[PCTINCREASE pourcentage) ) ;
```

Taille du fichier (tablespace)

Chemin d'accès du **fichier** dans lequel est stocké le tablespace
Ex : 'F:\oracle\oradata\data_tbs.dbf'

La création d'un tablespace : Syntaxe abrégé (qq options)

```
CREATE TABLESPACE nom_tablespace
DATAFILE 'chemin d'accès du fichier' SIZE valeur [K|M]
[MINIMUM EXTENT valeur [K|M] ]
```

Taille **minimale** d'une extension

```
[MAXIMUM EXTENT valeur [K|M] ]
```

Taille **maximale** d'une extension

```
[DEFAULT STORAGE
( [INITIAL valeur [K|M]]
[ NEXT valeur [K|M]]
[ MINEXTENTS nombre]
[ MAXEXTENTS nombre]
[ PCTINCREASE pourcentage] ) ;
```



```
CREATE TABLESPACE nom_tablespace
DATAFILE 'chemin d'accès du fichier' SIZE valeur [K|M]
[MINIMUM EXTENT valeur [K|M] ]
[ MAXIMUM EXTENT valeur [K|M] ]
[DEFAULT STORAGE
( [INITIAL valeur [K|M]]
[ NEXT valeur [K|M]]
[ MINEXTENTS nombre]
[ MAXEXTENTS nombre]
[ PCTINCREASE pourcentage] ) ;
```

Caractéristiques du stockage

- **INITIAL** : Taille de la première extension allouée lors de la création d'un segment
- **NEXT** : Taille de la deuxième extension du segment
- **MINEXTENTS** : Nombre d'extensions allouées à la création du segment
- **MAXEXTENTS** : Nombre maximal d'extensions pouvant être allouées au segment
- **PCTINCREASE** : Pourcentage d'accroissement de la taille des extensions, appliqué à partir de la troisième extension



La création d'un tablespace : Syntaxe global (toutes options)

- À la création d'un tablespace, vous indiquez le ou les fichiers de données, leurs tailles, leur mode d'agrandissement. La création du tablespace permet également de configurer la taille du bloc, les informations de journalisation ou les options de compression, etc. ; ces options seront détaillées au fur et à mesure que nous avancerons dans ce chapitre.
- L'instruction SQL qui permet de créer un **tablespace permanent** est :

```
CREATE {BIGFILE|SMALLFILE} TABLESPACE nom_tablespace
      [ DATAFILE ['nom_fichier'] [ SIZE integer {K|M|G|T} ]
        [ AUTOEXTEND {OFF | ON [ NEXT integer {K|M|G|T}]} ]

        [ MAXSIZE {UNLIMITED | integer {K|M|G|T}}] [,...]]
[BLOCKSIZE integer [ K ]]
[ {LOGGING | NOLOGGING} ] [FORCE LOGGING]
[ {ONLINE | OFFLINE} ] [FLASHBACK {ON | OFF}]
[ENCRYPTION [ USING 'algorithm']]
      [IDENTIFIED BY mot_de_passe] DEFAULT STORAGE ENCRYPT]
[ DEFAULT { COMPRESS { BASIC
              | FOR { OLTP | { QUERY | ARCHIVE } [ LOW | HIGH ] }
              | NOCOMPRESS} } ] ;
```

AUTOEXTEND	L'argument active ou désactive l'extension automatique d'un nouveau fichier de données ou temporaire. Si vous omettez cette clause, ces fichiers ne seront pas automatiquement étendus.
BIGFILE	Indique que le tablespace est créé avec un seul fichier pouvant contenir jusqu'à 2^{32} blocks. Pour un tablespace d'un block de 32k, vous pouvez stocker jusqu'à 128Tb.
SMALLFILE	Indique que le tablespace peut avoir un ou plusieurs fichiers de données. Aucun fichier ne peut contenir plus de 2^{22} blocks. Pour un tablespace d'un block de 32Kb, chaque fichier peut stocker jusqu'à 128Gb.
nom_fichier	Le ou les fichiers de données qui constituent le tablespace.
integer	Spécifie une taille en octets si vous ne précisez pas de suffixe pour définir une valeur en K, M, G, T.
K	Valeurs spécifiées pour préciser la taille en kilo-octets.
M	Valeurs spécifiées pour préciser la taille en mégaoctets.
G	Valeurs spécifiées pour préciser la taille en gigaoctets.
T	Valeurs spécifiées pour préciser la taille en téraoctets.

La création d'un tablespace : Syntaxe global (toutes options)

- L'instruction SQL qui permet de créer un **tablespace permanent** est (suite) :

```
CREATE {BIGFILE|SMALLFILE} TABLESPACE nom_tablespace
    [ DATAFILE ['nom_fichier'] [ SIZE integer {K|M|G|T} ]
      [ AUTOEXTEND {OFF | ON [ NEXT integer {K|M|G|T}]} ]

      [ MAXSIZE {UNLIMITED | integer {K|M|G|T}}] [,...]]
[BLOCKSIZE integer [ K ]]
[ {LOGGING | NOLOGGING} ] [FORCE LOGGING]
[ {ONLINE | OFFLINE} ] [FLASHBACK {ON | OFF}]
[ENCRYPTION [ USING 'algorithm']
  [IDENTIFIED BY mot_de_passe] DEFAULT STORAGE ENCRYPT]
[ DEFAULT { COMPRESS { BASIC
  | FOR { OLTP | { QUERY | ARCHIVE } [ LOW | HIGH ] }
  | NOCOMPRESS} } ] ;
```

NEXT

Définit la taille en octets du prochain incrément d'espace disque qui doit être alloué automatiquement au fichier lorsque davantage d'espace de stockage est requis.

MAXSIZE

Définit l'espace disque maximal autorisé pour l'extension automatique du fichier de données.

UNLIMITED

Définit une allocation d'espace illimitée pour le fichier.

BLOCKSIZE

Indique une taille de bloc non standard pour le tablespace.

LOGGING

Définit que la base de données effectue les journalisations pour toutes les opérations sur tous les index, tables et partitions contenus dans le tablespace. L'attribut de journalisation de niveau tablespace peut être modifié par les spécifications de journalisation au niveau table, index ou partition.

NOLOGGING

Définit que la base de données n'effectue pas de journalisation pour les « **INSERT /*+APPEND*/** » ou les chargements des données à l'aide de SQL*Loader ainsi que les opérations DDL sur toutes les créations d'index, tables et partitions contenus dans le tablespace. Les modifications de tables avec les options « **MOVE** » ou « **SPLIT** ».

La création d'un tablespace : Exemples

FORCE LOGGING	Force le travail en mode « LOGGING » pour toutes les opérations de la base de données, même si l'opération concernée est effectuée dans le mode « NOLOGGING ».	<pre> SYS@saphir>create tablespace tbsdata datafile 2 size 10m autoextend on next 10m maxsize 1g, 3 size 10m autoextend on next 10m maxsize 1g 4 force logging online flashback on; Tablespace créé. SYS@saphir>select file_name 2 from dba_data_files 3 where tablespace_name = 'TBSDATA'; FILE_NAME ----- /u02/donnees/oradata/SAPHIR/datafile/o1_mf_tbsdata_63mdggwt_.dbf /u02/donnees/oradata/SAPHIR/datafile/o1_mf_tbsdata_63mdgh16_.dbf SYS@saphir>ls -l /u02/donnees/oradata/SAPHIR/datafile ... -rw-r----- 1 oracle oinstall 10493952 jui 11 13:58 o1_mf_tbsdata_63mdggwt_.dbf -rw-r----- 1 oracle oinstall 10493952 jui 11 13:58 o1_mf_tbsdata_63mdgh16_.dbf ... </pre>
ONLINE	Permet de créer un tablespace mis à la disposition des utilisateurs qui ont reçu le droit d'y accéder immédiatement après sa création. Il s'agit du choix par défaut	
OFFLINE	Permet de créer un tablespace dans un état indisponible immédiatement après sa création.	
FLASHBACK	Indique que le tablespace peut être utilisé dans des opérations de récupération de type « FLASHBACK ».	
ENCRYPTION	Indique que le tablespace stocke les informations cryptées dans ces fichiers. Cette option doit être accompagnée de la clause de stockage : « DEFAULT STORAGE ENCRYPT ».	
COMPRESS	Indique que le stockage de segments dans le tablespace est compressé. Pour plus d'informations sur le stockage des segments, voir le module « La gestion du stockage ».	

- Dans l'exemple précédent, vous pouvez remarquer la création d'un tablespace avec deux fichiers de données, chacun d'une taille de 10M. L'espace de stockage défini pour chaque fichier de 10Mb est réservé sur disque.



À la création du tablespace, il faut veiller à ce que les tailles des fichiers mentionnés ne dépassent pas l'espace libre qui se trouve sur disque. En effet, Oracle réserve automatiquement l'espace précisé pour chaque fichier de données. S'il n'y a pas assez d'espace sur disque, Oracle projette la création du tablespace.

La création d'un tablespace : Exemples

- Vous pouvez également définir si le tablespace créé est **accessible** tout de suite après la création, comme celui créé dans l'exemple précédent.

```
SYS@saphir>create table table_01 tablespace tbsdata
2 as select * from cat;

Table créée.
```

- Le tablespace créé « **OFFLINE** » n'est pas accessible immédiatement après sa création ; il faut modifier son état pour pouvoir travailler avec un tel type de tablespace.

```
SYS@saphir>create tablespace tbsdata02 datafile
2 size 10m autoextend on next 10m maxsize 1g
3 force logging offline flashback on;

Tablespace créé.

SYS@saphir>create table table_02 tablespace tbsdata02 as
2 select * from cat;
select * from cat
          *
ERREUR à la ligne 2 :
ORA-01542: tablespace 'TBSDATA02' hors ligne ; impossible de lui affecter de l'espace

SYS@saphir>alter tablespace tbsdata02 online;

Tablespace modifié.

SYS@saphir>create table table_02 tablespace tbsdata02 as
2 select * from cat;

Table créée.
```

Le tablespace par défaut

- Lorsqu'un utilisateur crée un segment sans préciser de tablespace, Oracle stocke le segment dans le tablespace par défaut de l'utilisateur. Ce tablespace est défini grâce à la clause **DEFAULT TABLESPACE** des ordres **SQL CREATE USER** et **ALTER USER**.

```
SQL>create user testTab identified by test default tablespace USERS quota 10M on USERS;
```

- Tous les objets qui ne comportent pas, dans la syntaxe de création, les mentions de stockage dans un tablespace, sont **stockés automatiquement** dans le tablespace permanent par défaut de la base de données.
- Il faut prendre l'habitude de définir un tablespace permanent par défaut qui reçoive l'ensemble des objets créés sans mention de stockage.
- Vous pouvez définir le tablespace permanent par défaut de la base de données à l'aide de la commande SQL suivante :

```
ALTER DATABASE DEFAULT TABLESPACE nom_tablespace ;
```

```
SYS@saphir>alter database default tablespace tbsdata;
Base de données modifiée.

SYS@saphir>select property_value from database_properties
  2  where property_name like 'DEFAULT_PERMANENT_TABLESPACE';
PROPERTY_VALUE
-----
TBSDATA

SYS@saphir>create table table_03 as select * from cat;
Table créée.

SYS@saphir>select table_name, tablespace_name
  2  from dba_tables
  3  where tablespace_name = 'TBSDATA';
TABLE_NAME          TABLESPAC
-----
TABLE_3              TBSDATA
```


Le tablespace par défaut

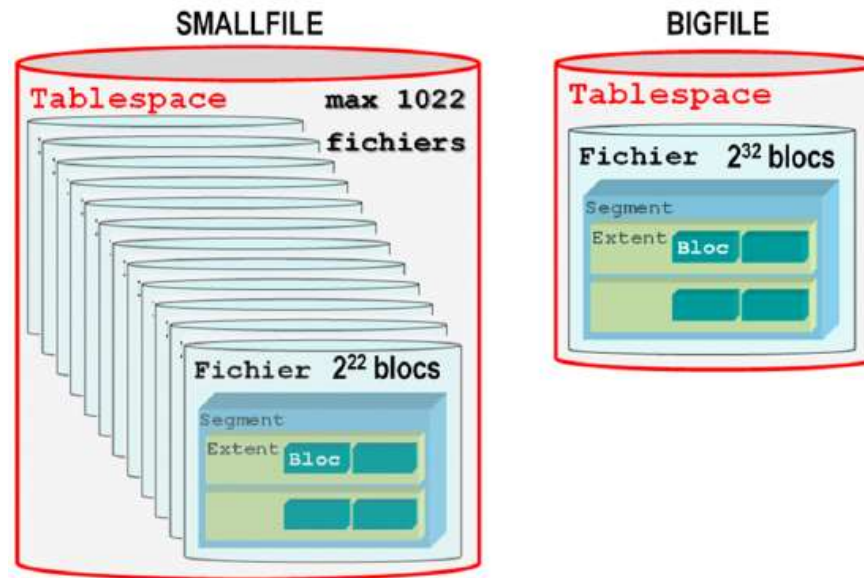
- La vue « **DATABASE_PROPERTIES** » permet d'afficher toutes les informations sur les **propriétés par défaut** d'une base de données comme : le nom global de la base de données, le tablespace permanent par défaut, le tablespace temporaire par défaut, le type de tablespace, le fuseau horaire, la langue, le territoire, ...
- Le **tablespace permanent par défaut** peut être défini lors de la création de la base de données, grâce à l'argument « **DEFAULT TABLESPACE** ».

```
SQL> create database "tpdba"
...
 7 datafile size 300m autoextend on
 8     next 10240k
 9     maxsize unlimited
10     extent management local
11 sysaux datafile size 120m autoextend on
12     next 10240k
13     maxsize unlimited
14 default temporary tablespace temp tempfile size 20m
15     autoextend on next 640k maxsize unlimited
16 undo tablespace "UNDOTBS1" datafile size 200m
17     autoextend on next 5120k maxsize unlimited
16 default tablespace app_users datafile size 200m
17     autoextend on next 5120k maxsize unlimited
...
```

- Comme vous pouvez le remarquer dans l'exemple précédent, la création de la base de données permet de créer en plus des quatre tablespaces « **SYSTEM** », « **SYSAUX** », « **UNDOTBS1** » et « **TEMP** », un autre tablespace permanent « **APP_USERS** » pour devenir le tablespace permanent par défaut de la base.

Les tablespaces BIGFILE & SMALLFILE

- Le tablespace de type « **SMALLFILE** » peut avoir **plusieurs fichiers de données**, au maximum **1022** fichiers, qui peuvent chacun contenir jusqu'à **2^{22}** blocs de données. Ainsi pour un tablespace construit avec un bloc de **32Kb**, vous pouvez avoir un espace de stockage par fichier de **128Gb**. La taille maximale de stockage pour un tablespace « **SMALLFILE** » est d'approximativement **128Tb** (il faut **1024** fichiers pour une taille exacte de **128Tb**).
- L'**inconvénient** de ce type de tablespace, c'est que lors d'un **point de contrôle (checkpoint)**, le processus « **DBWn** » écrit les blocs modifiés dans les fichiers de données correspondants et le processus « **CKPT** » met à jour l'en-tête de tous les fichiers de la base de données qui sont en mode lecture écriture. Ainsi plus il y a de fichiers **plus les checkpoints durent longtemps** même si ces fichiers n'ont pas de blocs de données modifiés.



Les tablespaces BIGFILE & SMALLFILE

- Le tablespace type « **BIGFILE** » est un tablespace avec un **seul fichier de données** qui peut contenir jusqu'à **2³²** blocs de données. Ainsi un fichier de données pour un tablespace construit avec des blocs de **32Kb** peut stocker jusqu'à **128Tb**. Ainsi ce type de tablespace **réduit le nombre de fichiers de données** en occurrence avec le **temps des checkpoints**.
- Lorsque vous créez votre base de données, vous pouvez spécifier le **type de tablespace par défaut** que vous souhaitez, à l'aide de la syntaxe :

```
SET DEFAULT {BIGFILE | SMALLFILE} TABLESPACE ...
```

- Sans précision explicitée, le type du tablespace « **SYSTEM** » est considéré comme le **type par défaut**. Après la création de la base de données vous pouvez également changer le type par défaut à l'aide de la syntaxe :

```
ALTER DATABASE SET DEFAULT {BIGFILE | SMALLFILE} TABLESPACE ;
```

- La vue « **DATABASE_PROPERTIES** » vous permet d'afficher le **type de tablespace par défaut** de votre base de données.

Les tablespaces BIGFILE & SMALLFILE

```
SYS@rubis>alter database set default bigfile tablespace;
```

Base de données modifiée.

```
SYS@rubis>select property_name, property_value
2  from database_properties
3  where property_name = 'DEFAULT_TBS_TYPE';
```

PROPERTY_NAME	PROPERTY_VALUE
DEFAULT_TBS_TYPE	BIGFILE

```
SYS@rubis>create tablespace tbsbig datafile
2  size 10m autoextend on next 10m;
```

Tablespace créé.

NEXT

```
SYS@rubis>create tablespace tbssmall datafile
```

```
2  size 10m autoextend on next 10m ,
3  size 10m autoextend on next 10m;
```

```
create tablespace tbssmall datafile
```

```
*
```

ERREUR à la ligne 1 :

ORA-32774: plusieurs fichiers ont été indiqués pour le tablespace BIGFILE TBSSMALL

```
SYS@rubis>create smallfile tablespace tbssmall datafile
```

```
2  size 10m autoextend on next 10m ,
3  size 10m autoextend on next 10m;
```

Tablespace créé.

```
SYS@rubis>select tablespace_name, bigfile, file_name
2  from dba_data_files join
3       dba_tablespaces using ( tablespace_name)
4  where tablespace_name in ( 'TBSBIG', 'TBSSMALL');
```

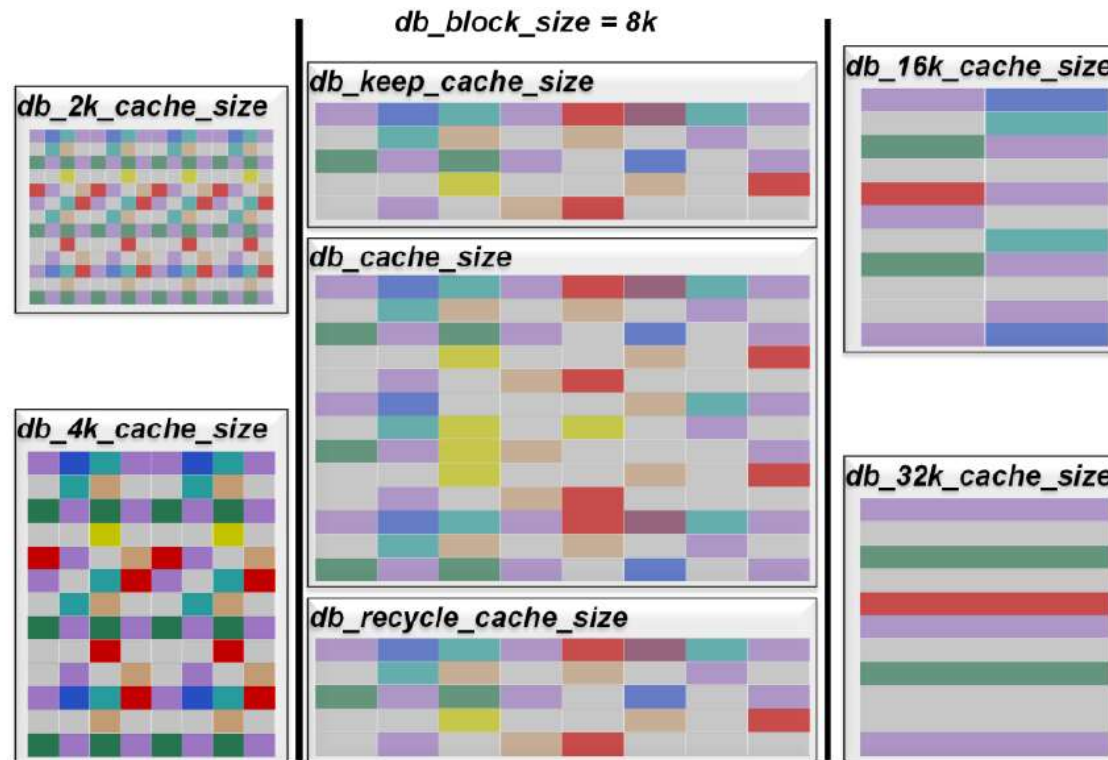
TABLESPACE	BIG	FILE_NAME
TBSBIG	YES	+GD_DONNEES/rubis/datafile/tbsbig.330.724090169
TBSSMALL	NO	+GD_DONNEES/rubis/datafile/tbssmall.329.724090417
TBSSMALL	NO	+GD_DONNEES/rubis/datafile/tbssmall.328.724090419



- Dans l'exemple précédent, vous pouvez voir la configuration de la base de données pour créer **par défaut** des tablespaces de type « **BIGFILE** ». Par la suite, la création du tablespace « **TBSBIG** » est effectuée avec succès mais la création d'un tablespace « **TBSSMALL** » avec plusieurs fichiers échoue. En précisant qu'il s'agit d'un tablespace de type « **SMALLFILE** » le tablespace est cette fois-ci créé.

La taille du bloc

- Pour **lire ou écrire** les données de la **base de données**, Oracle charge d'abord les blocs correspondants dans le **buffer cache** (cache de tampon). Ainsi il faut d'abord **paramétrer le buffer cache** pour pouvoir **recevoir les blocs** respectifs.



- Le paramètre qui vous permet de réserver de l'espace pour les blocs différents des blocs par défaut est : **DB_nK_CACHE_SIZE**.
- Les valeurs autorisées pour le « **n** » sont 2, 4, 8, 16 et 32 (certains systèmes d'exploitation ne supportent pas cette valeur). La valeur du bloc par défaut est « **DB_BLOCK_SIZE** », paramètre défini à la création de la base de données.

La taille du bloc

```
SYS@rubis>show parameter db_block_size
```

NAME	TYPE	VALUE
db_block_size	integer	8192

```
SYS@rubis>show parameter k_cache_size
```

NAME	TYPE	VALUE
db_16k_cache_size	big integer	0
db_2k_cache_size	big integer	0
db_32k_cache_size	big integer	0
db_4k_cache_size	big integer	0
db_8k_cache_size	big integer	0

```
SYS@rubis>alter system set db_32k_cache_size=16m;
```

Système modifié.

```
SYS@rubis>alter system set db_4k_cache_size=16m;
```

Système modifié.

```
SYS@rubis>show parameter k_cache_size
```

NAME	TYPE	VALUE
db_16k_cache_size	big integer	0
db_2k_cache_size	big integer	0
db_32k_cache_size	big integer	16M
db_4k_cache_size	big integer	16M
db_8k_cache_size	big integer	0

- L'exemple précédant montre le paramétrage de la mémoire pour pouvoir créer des tablespaces qui utilisent une taille de bloc de **4Kb** et **32Kb** en plus de la taille par défaut de **8Kb**.

La taille du bloc

```

SYS@rubis>create tablespace data_4k datafile size 10m autoextend on
2  next 10m , blocksize 4k;

Tablespace créé.

SYS@rubis>create tablespace data_32k datafile size 10m autoextend on
2  next 10m blocksize 32k;

Tablespace créé.

SYS@rubis>create tablespace data_2k datafile size 10m autoextend on
2  next 10m blocksize 2k;
create tablespace data_2k datafile
*
ERREUR à la ligne 1 :
ORA-29339: la taille de bloc de tablespace 2048 ne correspond pas aux tailles de blocs
configures

SYS@rubis>select tablespace_name, block_size/1024 "bs K", file_name
2  from dba_data_files join dba_tablespaces using ( tablespace_name)
3  where tablespace_name like 'DATA%K';

TABLESPAC bs K FILE_NAME
-----
DATA_4K      4 +GD_DONNEES/rubis/datafile/data_4k.327.724092783
DATA_32K     32 +GD_DONNEES/rubis/datafile/data_32k.326.724092799

```

- Les deux premières créations de tablespace sont acceptées par la base de données, et le paramétrage du **buffer cache** est déjà effectué. Par contre la dernière création est rejetée ; il faut d'abord **paramétrer le buffer cache** pour pouvoir recevoir les blocs de **2Kb**.

La taille du bloc



- Pour lire ou écrire les données de la base de données, Oracle **charge d'abord les blocs** correspondants dans le **buffer cache (cache de tampon)**.
- Si une **des zones** d'une taille de bloc non standard vient d'être **mise à zéro**, alors tous les objets des tablespaces qui ont **cette taille de bloc** ne peuvent plus être lus, comme c'est le cas dans l'exemple précédent.
- Une fois que la **zone mémoire** a été **initialisée**, les données sont à nouveau **accessibles**.

```
SYS@rubis>create table tdata_32k tablespace data_32k as select * from cat;
```

Table créée.

```
SYS@rubis>alter system set db_32k_cache_size=0;
```

Système modifié.

```
SYS@rubis>select * from tdata_32k;
```

```
select * from tdata_32k
```

*

ERREUR à la ligne 1 :

ORA-00379: aucun tampon libre dans le pool de tampons DEFAULT pour la taille de bloc (32 ko)

```
SYS@rubis>alter system set db_32k_cache_size=16m;
```

Système modifié.

```
SYS@rubis>select count(*) from tdata_32k;
```

COUNT(*)

```
-----
4854
```


Le tablespace temporaire

- Lors d'importantes **opérations de tri** (telles que **select distinct**, **union** et **create index**), Oracle a besoin de stocker dans la base de données, des informations concernant le tri des enregistrements, avant de retourner l'information aux utilisateurs. En raison de leur **nature dynamique**, ces espaces de tri ne devraient pas être stockés avec d'autres types de données.
- Lorsqu'un tablespace temporaire « **TEMPORARY** » est défini, un **segment de tri** est aussi **créé**. Celui-ci est capable de croître si nécessaire afin de pouvoir héberger toutes les opérations de tri de données, et existe jusqu'à ce que la base de données soit **fermée** puis **redémarrée**.



Vous ne pouvez pas créer de **tablespaces temporaires** avec une taille de bloc différente de la **taille du bloc par défaut** de la base de données. Les **tablespaces temporaires** ainsi que les **tablespaces système** sont toujours créés avec une taille de bloc égale au **bloc par défaut** de la base de données.

La syntaxe de création d'un tablespace temporaire est :

```
CREATE {BIGFILE|SMALLFILE} TEMPORARY TABLESPACE nom_tablespace
[ TEMPFILE ['nom_fichier'] [ SIZE integer {K|M|G|T} ]
[ AUTOEXTEND {OFF | ON [ NEXT integer {K|M|G|T}]}
[ MAXSIZE {UNLIMITED | integer {K|M|G|T}}] [,...]]
[{ONLINE | OFFLINE}];
```

TEMPORARY	Indique que le tablespace est de type temporaire.
TEMPFILE	Indique qu'il s'agit d'un fichier temporaire et pas d'un fichier de données.
REUSE	Définit la réutilisation du fichier temporaire s'il existe déjà.

Le tablespace temporaire

```

SYS@jaspe>create bigfile temporary tablespace temp01 tempfile
  2  size 10m autoextend on next 10m ;

Tablespace créé.

SYS@jaspe>create bigfile temporary tablespace temp02 tempfile
  2  size 10m autoextend on next 10m ;

Tablespace créé.

SYS@jaspe>create bigfile temporary tablespace temp03 tempfile
  2  size 10m autoextend on next 10m ;

Tablespace créé.

SYS@jaspe>select tablespace_name, file_name from dba_temp_files join
  2  dba_tablespaces using ( tablespace_name)
  3  where tablespace_name like 'TEMP0_';

TABLESPAC  FILE_NAME
-----
TEMP01      +GD_DONNEES/jaspe/tempfile/temp01.283.724094305
TEMP02      +GD_DONNEES/jaspe/tempfile/temp02.281.724094309
TEMP03      +GD_DONNEES/jaspe/tempfile/temp03.280.724094315

```

- Dans l'exemple précédent, vous pouvez observer la création de trois **tablespaces temporaires** de type « **BIGFILE** ». Les fichiers temporaires ainsi créés peuvent **s'agrandir automatiquement** par des tranches de **10Mb**.

Le tablespace temporaire

```
SYS@ambre>create temporary tablespace temp04
2 tempfile 'W:\DONNEES\ORADATA\AMBRE\DATAFILE\TEMP04.TMP'
3 size 10m autoextend on next 10m;
```

Tablespace créé.

```
SYS@ambre>drop tablespace temp04;
```

Tablespace supprimé.

```
SYS@ambre>create temporary tablespace temp04
2 tempfile 'W:\DONNEES\ORADATA\AMBRE\DATAFILE\TEMP04.TMP'
3 size 10m autoextend on next 10m;
create temporary tablespace temp04
*
```

ERREUR à la ligne 1 :

```
ORA-01119: échec de création du fichier de base de données
'W:\DONNEES\ORADATA\AMBRE\DATAFILE\TEMP04.TMP'
ORA-27038: le fichier créé existe déjà
OSD-04010: option <create> indiquée ; le fichier existe déjà
```

```
SYS@ambre>create temporary tablespace temp04
2 tempfile 'W:\DONNEES\ORADATA\AMBRE\DATAFILE\TEMP04.TMP'
3 size 20m reuse autoextend on next 10m;
```

Tablespace créé.



```
SYS@ambre>$dir W:\DONNEES\ORADATA\AMBRE\DATAFILE\TEMP04.TMP
Le volume dans le lecteur W s'appelle Nouveau nom
Le numéro de série du volume est C051-C97C

Répertoire de w:\donnees\oradata\AMBRE\DATAFILE

11/07/2010  18:03          20 979 712 TEMP04.TMP
...
```

- ❖ L'exemple précédent commence par la création d'un tablespace « **TEMP04** ». Ensuite on **détruit** le tablespace **sans effacer** le fichier temporaire. Vous pouvez remarquer que la création d'un tablespace ne peut pas être effectuée si le fichier correspondant **existe déjà**, et cela est valable pour tous les tablespaces. Vous pouvez utiliser l'argument « **REUSE** » pour demander à Oracle de **réutiliser le fichier existant** sur disque. En effet Oracle réutilise le nom du fichier temporaire et il le **dimensionne** au volume demandé.
- ❖ Cette démarche de **rattachement d'un fichier temporaire** est utilisée dans le cas de perte du fichier de contrôle et la création à l'aide du script de sauvegarde.

Le groupe tablespaces temporaires

- Certains utilisateurs d'une base de données Oracle peuvent avoir besoin de volumes de stockage temporaire beaucoup plus grands que ceux de tous les autres utilisateurs de l'application. Dans ce cas, vous pouvez créer plusieurs tablespaces temporaires « **TEMPORARY** » pour distribuer les espaces de stockages des utilisateurs ayant des besoins semblables sur les mêmes tablespaces.
- À partir de la version Oracle 10g, il est possible de créer des groupes de tablespaces temporaires pour **équilibrer les charges et diminuer les contentions d'accès** aux segments temporaires. Un **groupe de tablespaces temporaires** est géré par la base de données comme un **seul tablespace temporaire** ; vous pouvez ainsi définir que c'est le **tablespace temporaire par défaut**.
- La syntaxe de création d'un groupe de tablespaces temporaires est en effet uniquement un **assignement d'un tablespace temporaire** existant au **nom du groupe** qui va être créé, comme suit :

```
ALTER TABLESPACE nom TABLESPACE GROUP nom_groupe;
```

- Les trois tablespaces temporaires ont été ajoutés au groupe « **NGRP_TEMP** » ; à présent le nom groupe peut être utilisé à la place du nom d'un **tablespace temporaire**.

```
SYS@jaspe>alter tablespace temp01 tablespace group ngrp_temp;
Tablespace modifié.

SYS@jaspe>alter tablespace temp02 tablespace group ngrp_temp;
Tablespace modifié.

SYS@jaspe>alter tablespace temp03 tablespace group ngrp_temp;
Tablespace modifié.

SYS@jaspe>select * from dba_tablespace_groups;

GROUP_NAME                                TABLESPAC
-----
NGRP_TEMP                                TEMP01
NGRP_TEMP                                TEMP02
NGRP_TEMP                                TEMP03
```


Le groupe tablespaces temporaires

- Pour **retirer un tablespace** d'un groupe de tablespaces temporaires il suffit d'utiliser la syntaxe suivante :

```
ALTER TABLESPACE nom TABLESPACE GROUP ' ' ;
```

- Vous pouvez maintenant définir que le **groupe de tablespaces temporaires** « **NGRP_TEMP** » comme le **tablespace temporaire par défaut** de la base.

```
SYS@jaspe>alter database default temporary tablespace ngrp_temp;

Base de données modifiée.

SYS@jaspe>select property_value from database_properties
  2  where property_name like 'DEFAULT_TEMP_TABLESPACE';

PROPERTY_VALUE
-----
NGRP_TEMP
```

- Dans l'exemple précédent, vous pouvez remarquer la définition du tablespace temporaire par défaut avec l'utilisation d'un groupe de tablespaces « **NGRP_TEMP** », et l'utilisation de la vue « **DATABASE_PROPERTIES** » pour retrouver la valeur du tablespace temporaire par défaut de la base de données.

Le tablespace undo

- Toutes les **données d'annulation** sont stockées dans un tablespace spécial appelé « **UNDO** ». Lorsque vous créez un tablespace « **UNDO** », Oracle gère le stockage, la rétention et l'emploi de l'espace pour les données de rollback par l'intermédiaire de la fonction **SMU (System-Managed Undo)**. **Aucun objet permanent n'est placé dans le tablespace undo.**
- Pour pouvoir créer un **tablespace undo** à la **création de la base de données**, il faut prendre soin d'initialiser le paramètre :

UNDO_MANAGEMENT=AUTO

- À partir de la version Oracle 9i, il est fortement conseillé d'utiliser la **gestion automatique** des segments d'annulation. À partir de la version Oracle 10g, il est impératif d'utiliser ce mode gestion, sans quoi une grande partie des fonctionnalités de la base de données sont inutilisables.
- Rappelez-vous, la syntaxe SQL de création de la base de données comporte d'abord la création d'un tablespace « **SYSTEM** » et d'un tablespace « **SYSAUX** », ainsi que la création des tablespaces « **TEMP** » et « **UNDO** ».



Lors de la création de la base de données, si vous ne précisez pas de **tablespace undo**, Oracle crée un tablespace undo appelé « **SYS_UNDOTS** ». Le paramètre « **UNDO_TABLESPACE** » doit avoir la valeur « **SYS_UNDOTS** », sinon la base de données ne pourra pas être créée.

Le tablespace undo

- La syntaxe de création d'un tablespace **UNDO** est :

```
CREATE {BIGFILE|SMALLFILE} UNDO TABLESPACE nom_tablespace
      [ DATAFILE ['nom_fichier'] [ SIZE integer {K|M|G|T} ]
        [ AUTOEXTEND {OFF | ON [ NEXT integer {K|M|G|T}]}
          [ MAXSIZE {UNLIMITED | integer {K|M|G|T}}] ] [, ...]]
      [{ONLINE | OFFLINE}] ;
```

UNDO Indique que le tablespace est de type undo.

- Attention, les tablespaces pérennes et temporaires **par défaut** sont attribués à l'aide de la syntaxe « **ALTER DATABASE DEFAULT ...** » ; ce n'est pas le cas pour le tablespace de type « **UNDO** ». Le tablespace « **UNDO** » **par défaut** est celui qui est précisé dans le paramètre « **UNDO_TABLESPACE** ».

Le tablespace undo

```
SYS@rubis>show parameter undo_tablespace
```

NAME	TYPE	VALUE
undo_tablespace	string	UNDOTBS1

```
SYS@rubis>create bigfile undo tablespace undo datafile
2 size 512m autoextend on next 512m;
```

Tablespace créé.

```
SYS@rubis>select contents, file_name
```

```
2 from dba_data_files join
3 dba_tablespaces using ( tablespace_name)
4 where tablespace_name like 'UNDO';
```

CONTENTS	FILE_NAME
UNDO	+GD_DONNEES/rubis/datafile/undo.325.724104703

```
SYS@rubis>alter system set undo_tablespace =undo;
```

Système modifié.

```
SYS@rubis>show parameter undo_tablespace
```

NAME	TYPE	VALUE
undo_tablespace	string	UNDO



- Comme pour les tablespaces système et pour les tablespaces temporaires, le tablespace **UNDO** doit être créé avec la **taille de bloc par défaut**.
- La base de données Oracle utilise pour ses fichiers de gestion la taille de bloc par défaut définie lors de la création de cette base de données.

Gestion des tablespaces permanents

Modification / Renommer :

(Version 10G et plus)

- **Syntaxe :** `ALTER TABLESPACE ancien_nom RENAME TO nouveau_nom;`
- **Exemple :** `ALTER TABLESPACE deftbs RENAME TO tbsdef;`
- Les tablespaces **SYSTEM** et **SYSAUX**, ainsi que les tablespaces **OFFLINE**, ne peuvent pas être renommés.

Modification / Ajout d'un fichier de données :

- **Syntaxe :** `ALTER TABLESPACE nom ADD DATAFILE spécification_fichier[,...];`
- **Exemple :** `ALTER TABLESPACE data ADD DATAFILE 'f:\oradata\orcl2\data02.dbf' SIZE 100M AUTOEXTEND ON NEXT 10M MAXSIZE 500M ;`
- Utilisée pour allouer de l'**espace supplémentaire** sur un autre disque que le disque actuellement utilisé par le tablespace (sinon, autant modifier la taille du fichier de données existant).
- Interdite pour un tablespace **BIGFILE**.

Gestion des tablespaces permanents

Modification / Modifier la taille d'un fichier de données :

- S'effectue avec l'ordre SQL **ALTER TABLESPACE (BIGFILE)** ou **ALTER DATABASE**
- **Syntaxe :** (**RESIZE** donne la nouvelle taille à la hausse ou à la baisse)

ALTER TABLESPACE nom_tablespace_bigfile RESIZE valeur [K|M|G|T];

ALTER DATABASE DATAFILE 'nom_complet' [...] RESIZE valeur [K|M|G|T];

Exemple :

- Tout type de tablespace :

ALTER DATABASE DATAFILE 'f:\oradata\orcl2\data02.dbf' RESIZE 200M ;

- Tablespace **BIGFILE** uniquement :

ALTER TABLESPACE je_suis_gros RESIZE 1T ;

Gestion des tablespaces permanents

Modification / Modifier l'extension automatique d'un fichier de données :

- S'effectue avec l'ordre SQL **ALTER TABLESPACE (BIGFILE)** ou **ALTER DATABASE**
- **Syntaxe :**
ALTER TABLESPACE nom_tablespace_bigfile clause_auto_extension;

ALTER DATABASE DATAFILE 'nom_complet' [,..] clause_auto_extension;

Exemple :

- Tout type de tablespace :
 - **Désactivation** : **ALTER DATABASE DATAFILE 'f:\oradata\orcl2\data02.dbf' AUTOEXTEND OFF ;**
 - **Activation** : **ALTER DATABASE DATAFILE 'f:\oradata\orcl2\data02.dbf' AUTOEXTEND ON NEXT 200M MAXSIZE 800M ;**
- Tablespace **BIGFILE** uniquement :
 - **Désactivation** : **ALTER TABLESPACE je_suis_gros AUTOEXTEND OFF ;**
 - **Activation** : **ALTER TABLESPACE je_suis_gros AUTOEXTEND ON NEXT 1G MAXSIZE 100G ;**

Gestion des tablespaces permanents

Mise hors service d'un tablespace (OFFLINE) :

- **Commande** : `ALTER TABLESPACE nom_tablespace OFFLINE [NORMAL|IMMEDIATE];`
 - **OFFLINE NORMAL** : le tablespace est **mis hors service** lorsque tous les utilisateurs du tablespace ont fini leurs transactions en cours.
 - **OFFLINE IMMEDIATE** : le tablespace est **mis hors service** même si les transactions en cours ne sont pas terminées.
- La désactivation est nécessaire pour certaines opérations d'administration (déplacer un fichier de données, rendre les données inaccessibles).
- Le tablespace **SYSTEM** ne peut pas être **OFFLINE**.
- Le tablespace **SYSAUX** peut être passé **OFFLINE** mais certaines fonctionnalités risquent de ne plus fonctionner.

Mise en service d'un tablespace (ONLINE) :

- **Commande** : `ALTER TABLESPACE nom_tablespace ONLINE;`
- Le statut **ONLINE** / **OFFLINE** est **conservé lors de l'arrêt**.

Gestion des tablespaces permanents

Modification / renommer ou déplacer un fichier de données :

- S'effectue avec l'ordre SQL :
 - **ALTER TABLESPACE** : la base **OPEN**, le tablespace **OFFLINE**
 - **ALTER DATABASE** : la base **MOUNT**, le tablespace **OFFLINE**
- **Remarque** : ces deux ordres SQL ne manipulent pas physiquement le fichier, ils se contentent de mettre à jour le fichier de contrôle. Le fichier de données doit être renommé/copié/déplacé à l'aide d'une commande du système d'exploitation, **avant d'exécuter l'ordre SQL**.
- **Syntaxe** :
 - **ALTER TABLESPACE** nom **RENAME DATAFILE** 'ancien_nom_complet' **TO** 'nouveau_nom_complet';
 - **ALTER DATABASE** **RENAME FILE** 'ancien_nom_complet' **TO** 'nouveau_nom_complet';
- **Renommer** un fichier de données est surtout utilisé pour **déplacer le fichier**.



Gestion des tablespaces permanents

Modification / renommer ou déplacer un fichier de données :

Méthode 1 :

- 1) Se connecter en tant que DBA : `SQL> CONNECT / as sysdba`
- 2) Passer la tablespace OFFLINE : `SQL> ALTER TABLESPACE data OFFLINE;`
- 3) Par une commande du système d'exploitation, renommer, copie ou déplacer le fichier :
 - `SQL> HOST copy e:\oradata\orcl2\data02.dbf f:\oradata\orcl2\data02.dbf`
 - `SQL> HOST del e:\oradata\orcl2\data02.dbf`
- 4) Exécuter l'ordre SQL ALTER TABLESPACE : `SQL> ALTER TABLESPACE data RENAME DATAFILE 'e:\oradata\orcl2\data02.dbf' TO 'f:\oradata\orcl2\data02.dbf';`
- 5) Repasser le tablespace ONLINE : `SQL> ALTER TABLESPACE data ONLINE;`



Gestion des tablespaces permanents

Modification / renommer ou déplacer un fichier de données :

Méthode 2 :

- 1) Se connecter en tant que DBA : SQL> CONNECT / as sysdba
- 2) Passer la base de données en état MOUNT : SQL> SHUTDOWN IMMEDIATE
SQL> STARTUP MOUNT
- 3) Par une commande du système d'exploitation, renommer, copie ou déplacer le fichier:
 - SQL> HOST copy e:\oradata\orcl2\data02.dbf -f:\oradata\orcl2\data02.dbf
 - SQL> HOST del e:\oradata\orcl2\data02.dbf
- 4) Exécuter l'ordre SQL ALTER DATABASE : SQL> ALTER DATABASE RENAME FILE 'e:\oradata\orcl2\data02.dbf'
TO 'f:\oradata\orcl2\data02.dbf';
- 5) Ouvrir la base de données : SQL> ALTER DATABASE OPEN;

Gestion des tablespaces permanents

Modification des caractéristiques du stockage :

```
ALTER TABLESPACE nom_tablespace
[DEFAULT STORAGE
([INITIAL valeur [K|M]]
[NEXT valeur [K|M]]
[MINEXTENTS nombre]
[MAXEXTENTS nombre]
[PCTINCREASE pourcentage] ) ;
```



```
([INITIAL valeur [K|M]]
[NEXT valeur [K|M]]
[MINEXTENTS nombre]
[MAXEXTENTS nombre]
[PCTINCREASE pourcentage] ) ;
```

Caractéristiques du stockage

- **INITIAL** : Taille de la première extension allouée lors de la création d'un segment
- **NEXT** : Taille de la deuxième extension du segment
- **MINEXTENTS** : Nombre d'extensions allouées à la création du segment
- **MAXEXTENTS** : Nombre maximal d'extensions pouvant être allouées au segment
- **PCTINCREASE** : Pourcentage d'accroissement de la taille des extensions, appliqué à partir de la troisième extension

Gestion des tablespaces permanents

Tablespace en lecture seule :

ALTER TABLESPACE nom_tablespace **READ ONLY ;**

READ ONLY / READ WRITE :

ALTER TABLESPACE nom {**READ ONLY | READ WRITE** } ;



Lecture seule :

- Empêche toute opération d'écriture sur les fichiers de données.
- Un tablespace peut être créé pour recevoir des tables de type **historique** dont les données ne doivent jamais être **modifiées**. Il est possible de **protéger** une telle structure en attribuant au tablespace le mode **lecture seule**.

Autres opérations :

LOGGING/NOLOGGING :

ALTER TABLESPACE nom **LOGGING | NOLOGGING ;**

FORCE LOGGING :

ALTER TABLESPACE nom [**NO**] **FORCE LOGGING ;**

FLASHBACK ON / OFF

ALTER TABLESPACE nom **FLASHBACK ON | OFF ;**

Gestion des tablespaces permanents

Suppression d'un tablespace :

- L'ordre **SQL DROP TABLESPACE** ;

- **Syntaxe :**

DROP TABLESPACE nom [INCLUDING CONTENTS [AND DATAFILES] [CASCADE CONSTRAINTS]] ;

- **INCLUDING CONTENTS** : force la suppression préalable des segments qui y sont stockés.
- **AND DATAFILES** : supprime les fichiers physiques du tablespace.
- **CASCADE CONSTRAINTS** : supprime les contraintes d'intégrité référentielle définies sur des tables hors du tablespace et qui référencent des tables à l'intérieur du tablespace.

- **Exemple :** **DROP TABLESPACE data INCLUDING CONTENTS AND DATAFILES ;**

- **Remarques :**

- C'est un ordre **DDL**, pas de **ROLLBACK**.
- Le tablespace **SYSTEM** et le **tablespace permanent par défaut** ne peuvent pas être supprimés.
- Il est recommandé de passer le tablespace **OFFLINE** avant de le **supprimer**.

Obtenir des informations sur les tablespaces

Obtenir des informations sur les tablespaces

Vous pouvez obtenir des informations sur les tablespaces et les fichiers de données en interrogeant les éléments suivants :

- **Tablespaces :**
 - DBA_TABLESPACES
 - V\$TABLESPACE
- **Informations sur le fichier de données :**
 - DBA_DATA_FILES
 - V\$DATAFILE
- **Informations sur les fichiers temporaires :**
 - DBA_TEMP_FILES
 - V\$TEMPFILE

Obtenir des informations sur les tablespaces

Vue dba_tablespaces :

```
SQL> desc dba_tablespaces
Nom
-----
TABLESPACE_NAME      NOT NULL VARCHAR2(30)
BLOCK_SIZE           NOT NULL NUMBER
INITIAL_EXTENT        NUMBER
NEXT_EXTENT          NUMBER
MIN_EXTENTS          NOT NULL NUMBER
MAX_EXTENTS          NUMBER
MAX_SIZE             NUMBER
PCT_INCREASE         NUMBER
MIN_EXTLEN           NUMBER
STATUS               VARCHAR2(9)
CONTENTS             VARCHAR2(21)
LOGGING              VARCHAR2(9)
FORCE_LOGGING        VARCHAR2(3)
EXTENT_MANAGEMENT    VARCHAR2(10)
ALLOCATION_TYPE       VARCHAR2(9)
PLUGGED_IN           VARCHAR2(3)
SEGMENT_SPACE_MANAGEMENT VARCHAR2(6)
DEF_TAB_COMPRESSION  VARCHAR2(8)
RETENTION            VARCHAR2(11)
BIGFILE              VARCHAR2(3)
PREDICATE_EVALUATION VARCHAR2(7)
ENCRYPTED             VARCHAR2(3)
COMPRESS_FOR         VARCHAR2(30)
DEF_INMEMORY         VARCHAR2(8)
DEF_INMEMORY_PRIORITY VARCHAR2(8)
DEF_INMEMORY_DISTRIBUTE VARCHAR2(15)
DEF_INMEMORY_COMPRESSION VARCHAR2(17)
DEF_INMEMORY_DUPLICATE VARCHAR2(13)
SHARED              VARCHAR2(13)
DEF_INDEX_COMPRESSION VARCHAR2(8)
INDEX_COMPRESS_FOR   VARCHAR2(13)
DEF_CELLMEMORY       VARCHAR2(14)
DEF_INMEMORY_SERVICE VARCHAR2(12)
DEF_INMEMORY_SERVICE_NAME VARCHAR2(1000)
LOST_WRITE_PROTECT   VARCHAR2(7)
CHUNK_TABLESPACE     VARCHAR2(1)
```

DBA_TABLESPACES :

- **TABLESPACE_NAME** : nom de tablespace
- **CONTENTS** : type de tablespace (PERMANENT ou TEMPORARY ou UNDO)
- **EXTENT_MANAGEMENT** : (DICTIONARY ou LOCAL)
- **ALLOCATION_TYPE** : gestion des extensions USER (utilisateur), SYSTEM (locale, automatique) ou UNIFORM (locale et uniforme)
- **STATUS** : ONLINE, OFFLINE ou READ ONLY
- **BLOCK_SIZE** : taille du bloc du tablespace
- **LOGGING** : mode de journalisation (LOGGING ou NOLOGGING)
- **FORCE_LOGGING** : YES ou NO
- **SEGMENT_SPACE_MANAGEMENT** : gestion d'espace libre (AUTO, MANUAL)
- **BIGFILE** : YES ou NO

Obtenir des informations sur les tablespaces

Vues dba_data_files & dba_temp_files :

```
SQL> desc dba_data_files
Nom                                NULL ?    Type
-----
FILE_NAME                          VARCHAR2(513)
FILE_ID                            NUMBER
TABLESPACE_NAME                    VARCHAR2(30)
BYTES                              NUMBER
BLOCKS                             NUMBER
STATUS                             VARCHAR2(9)
RELATIVE_FNO                       NUMBER
AUTOEXTENSIBLE                    VARCHAR2(3)
MAXBYTES                           NUMBER
MAXBLOCKS                         NUMBER
INCREMENT_BY                      NUMBER
USER_BYTES                        NUMBER
USER_BLOCKS                       NUMBER
ONLINE_STATUS                     VARCHAR2(7)
LOST_WRITE_PROTECT                VARCHAR2(7)

SQL> desc dba_temp_files
Nom                                NULL ?    Type
-----
FILE_NAME                          VARCHAR2(513)
FILE_ID                            NUMBER
TABLESPACE_NAME                    VARCHAR2(30)
BYTES                              NUMBER
BLOCKS                             NUMBER
STATUS                             VARCHAR2(7)
RELATIVE_FNO                       NUMBER
AUTOEXTENSIBLE                    VARCHAR2(3)
MAXBYTES                           NUMBER
MAXBLOCKS                         NUMBER
INCREMENT_BY                      NUMBER
USER_BYTES                        NUMBER
USER_BLOCKS                       NUMBER
SHARED                            VARCHAR2(13)
INST_ID                           NUMBER
```

DBA_DATA_FILES et DBA_TEMP_FILES :

- **FILE_NAME** : nom du fichier de données (chemin complet)
- **FILE_ID** : Identifiant du fichier de données
- **TABLESPACE_NAME** : le nom du tablespace auquel le fichier appartient
- **BYTES** : taille du fichier en octets
- **BLOCKS** : taille du fichier en bloc Oracle
- **STATUS** : statut du fichier (INVALID ou AVAILABLE)
- **RELATIVE_FNO** : numéro relatif du fichier par rapport au tablespace
- **AUTOEXTENSIBLE** : indicateur d'autoextensibilité (YES, NO)
- **MAXBYTES** : taille maximale du fichier en octets
- **MAXBLOCKS** : taille maximale du fichier en bloc Oracle
- **INCREMENT_BY** : taille de l'incrément de l'autoextensibilité en bloc Oracle
- **USER_BYTES** : taille utile du fichier en octets (taille du fichier-la taille de l'entête)
- **USER_BLOCKS** : taille utile du fichier en blocs ORACLE (-les blocs d'entête)

Obtenir des informations sur les tablespaces

Vues v\$tablespace, v\$datafile & v\$tempfile :

v\$datafile :

```
SQL> desc v$datafile
```

Nom	NULL ?	Type
FILE#		NUMBER
CREATION_CHANGE#		NUMBER
CREATION_TIME		DATE
TS#		NUMBER
RFILE#		NUMBER
STATUS		VARCHAR2(7)
ENABLED		VARCHAR2(10)
CHECKPOINT_CHANGE#		NUMBER
CHECKPOINT_TIME		DATE
UNRECOVERABLE_CHANGE#		NUMBER
UNRECOVERABLE_TIME		DATE
LAST_CHANGE#		NUMBER
LAST_TIME		DATE
OFFLINE_CHANGE#		NUMBER
ONLINE_CHANGE#		NUMBER
ONLINE_TIME		DATE
BYTES		NUMBER
BLOCKS		NUMBER
CREATE_BYTES		NUMBER
BLOCK_SIZE		NUMBER
NAME		VARCHAR2(513)
PLUGGED_IN		NUMBER
BLOCK1_OFFSET		NUMBER
AUX_NAME		VARCHAR2(513)
FIRST_NONLOGGED_SCN		NUMBER
FIRST_NONLOGGED_TIME		DATE
FOREIGN_DBID		NUMBER
FOREIGN_CREATION_CHANGE#		NUMBER
FOREIGN_CREATION_TIME		DATE
PLUGGED_READONLY		VARCHAR2(3)
PLUGIN_CHANGE#		NUMBER
PLUGIN_RESETLOGS_CHANGE#		NUMBER
PLUGIN_RESETLOGS_TIME		DATE
CON_ID		NUMBER

v\$tablespace :

```
SQL> desc v$tablespace
```

Nom	NULL ?	Type
TS#		NUMBER
NAME		VARCHAR2(30)
INCLUDED_IN_DATABASE_BACKUP		VARCHAR2(3)
BIGFILE		VARCHAR2(3)
FLASHBACK_ON		VARCHAR2(3)
ENCRYPT_IN_BACKUP		VARCHAR2(3)
CON_ID		NUMBER

v\$tempfile :

```
SQL> desc v$tempfile
```

Nom	NULL ?	Type
FILE#		NUMBER
CREATION_CHANGE#		NUMBER
CREATION_TIME		DATE
TS#		NUMBER
RFILE#		NUMBER
STATUS		VARCHAR2(7)
ENABLED		VARCHAR2(10)
BYTES		NUMBER
BLOCKS		NUMBER
CREATE_BYTES		NUMBER
BLOCK_SIZE		NUMBER
NAME		VARCHAR2(513)
CON_ID		NUMBER

Obtenir des informations sur les tablespaces

Vues v\$tablespace, v\$datafile & v\$tempfile :



V\$TABLESPACE :

- **TS# :** numéro du tablespace
- **NAME :** nom du tablespace
- **BIGFILE :** YES ou NO

V\$DATAFILE et V\$TEMPFILE :

- **TS# :** numéro du tablespace auquel le fichier de données appartient
- **FILE# :** Identifiant du fichier de données
- **NAME :** chemin complet du fichier de données
- **STATUS :** (OFFLINE, ONLINE, SYSTEM ou RECOVER)
- **ENABLED :** disponibilité du fichier de données (DISABLED, READ ONLY ou READ WRITE)
- **BYTES :** taille du fichier en octets
- **CREATE_BYTES :** taille du fichier à sa création en octets
- **BLOCKS :** taille du fichier à sa création en bloc Oracle
- **BLOCKS_SIZE :** taille du bloc du fichier de données
- **CREATION_TIME :** date et heure de création du fichier
- **CHECKPOINT_CHANGE# :** Numéro SCN du dernier point de reprise (inexistant en V\$TEMPFILE)
- **CHECKPOINT_TIME :** date et heure de dernier point de reprise (inexistant en V\$TEMPFILE)

Obtenir des informations sur les tablespaces

Vues dba_segments & dba_free_space :

```
SQL> desc dba_segments
Nom                                     NULL ?   Type
-----
OWNER                                   VARCHAR2(128)
SEGMENT_NAME                           VARCHAR2(128)
PARTITION_NAME                         VARCHAR2(128)
SEGMENT_TYPE                           VARCHAR2(18)
SEGMENT_SUBTYPE                        VARCHAR2(10)
TABLESPACE_NAME                        VARCHAR2(30)
HEADER_FILE                            NUMBER
HEADER_BLOCK                           NUMBER
BYTES                                  NUMBER
BLOCKS                                 NUMBER
EXTENTS                                NUMBER
INITIAL_EXTENT                         NUMBER
NEXT_EXTENT                           NUMBER
MIN_EXTENTS                            NUMBER
MAX_EXTENTS                            NUMBER
MAX_SIZE                               NUMBER
RETENTION                             VARCHAR2(7)
MINRETENTION                           NUMBER
PCT_INCREASE                           NUMBER
FREELISTS                              NUMBER
FREELIST_GROUPS                        NUMBER
RELATIVE_FNO                           NUMBER
BUFFER_POOL                            VARCHAR2(7)
FLASH_CACHE                            VARCHAR2(7)
CELL_FLASH_CACHE                       VARCHAR2(7)
INMEMORY                               VARCHAR2(8)
INMEMORY_PRIORITY                       VARCHAR2(8)
INMEMORY_DISTRIBUTE                     VARCHAR2(15)
INMEMORY_DUPLICATE                      VARCHAR2(13)
INMEMORY_COMPRESSION                   VARCHAR2(17)
CELLMEMORY                             VARCHAR2(24)
```

```
SQL> desc dba_free_space
Nom                                     NULL ?   Type
-----
TABLESPACE_NAME                        VARCHAR2(30)
FILE_ID                                NUMBER
BLOCK_ID                               NUMBER
BYTES                                  NUMBER
BLOCKS                                 NUMBER
RELATIVE_FNO                           NUMBER
```

DBA_SEGMENTS :

- **OWNER** : nom du propriétaire du segment
- **SEGMENT_NAME** : nom du segment
- **SEGMENT_TYPE** : type de segment (TABLE, INDEX, ROLLBACK, TYPE2 UNDO, etc.)
- **TABLESPACE_NAME** : nom du tablespace qui contient le segment
- **BYTES** : taille du segment en octets
- **BLOCKS** : taille du segment en blocs Oracle
- **EXTENTS** : nombre d'extension alloué au segment
- **INITIAL_EXTENT** : taille initial du segment

DBA_FREE_SPACE :

- **TABLESPACE_NAME** : nom du tablespace qui contient l'extension libre
- **FILE_ID** : identifiant du fichier qui contient l'extension libre
- **BLOCK_ID** : numéro du bloc de l'extension libre
- **BYTES** : taille de l'extension libre en octets
- **BLOCKS** : taille de l'extension libre en blocs Oracle

Obtenir des informations sur les tablespaces

Vue dba_extents :

```
SQL> desc dba_extents
  Nom                                NULL ?    Type
-----
OWNER                                VARCHAR2(128)
SEGMENT_NAME                         VARCHAR2(128)
PARTITION_NAME                       VARCHAR2(128)
SEGMENT_TYPE                         VARCHAR2(18)
TABLESPACE_NAME                      VARCHAR2(30)
EXTENT_ID                           NUMBER
FILE_ID                             NUMBER
BLOCK_ID                             NUMBER
BYTES                                NUMBER
BLOCKS                               NUMBER
RELATIVE_FNO                         NUMBER
```

DBA_EXTENTS :

- **OWNER** : nom du propriétaire du segment auquel l'extension appartient
- **SEGMENT_NAME** : nom du segment auquel l'extension appartient
- **SEGMENT_TYPE** : type de segment (TABLE, INDEX, ROLLBACK, TYPE2 UNDO, etc.)
- **TABLESPACE_NAME** : nom du tablespace qui contient l'extension
- **EXTENT_ID** : numéro de l'extension (0 pour la première)
- **FILE_ID** : identifiant du fichier qui contient l'extension
- **BLOCK_ID** : numéro du premier bloc de l'extension
- **BYTES** : taille de l'extension en octets
- **BLOCKS** : taille de l'extension en blocs Oracle

En complément, plusieurs vues possèdent une colonne **TABLESPACE** indiquant le nom du tablespace de stockage, par exemple **DBA_INDEXES** et **DBA_TABLES**

Gérer des tablespaces à l'aide d'OMF

Gérer des tablespaces à l'aide d'OMF

- Le paramètre d'initialisation **DB_CREATE_FILE_DEST** est défini lors de la configuration d'Oracle-Managed Files (OMF) pour la création de tablespaces. La clause **DATAFILE** n'est pas requise.
- Tous les fichiers de données sont créés **automatiquement** et leur emplacement est défini par **DB_CREATE_FILE_DEST**. Le nom du fichier est généré automatiquement par le serveur Oracle (**ora_tbs1_2ixfh90q.dbf**).

Gérer des tablespaces à l'aide d'OMF

- Définissez le paramètre **DB_CREATE_FILE_DEST** en utilisant l'une des méthodes suivantes :
 - Fichier de paramètres d'initialisation
 - Définition dynamique à l'aide de la commande **ALTER SYSTEM**

```
ALTER SYSTEM SET
db_create_file_dest = '/u01/oradata/dba01';
```

- Lorsque vous créez le tablespace :
 - le fichier de données est automatiquement créé dans **DB_CREATE_FILE_DEST**,
 - la taille par défaut est de 100 Mo,
 - La valeur **UNLIMITED** est affectée à **AUTOEXTEND**.

Gérer des tablespaces à l'aide d'OMF

Gérer des tablespaces à l'aide d'OMF

■ Créer des tablespaces à l'aide d'OMF

- La création de tablespaces avec OMF ne requiert pas de clause **DATAFILE**.
- Dans ce cas, la taille par défaut des fichiers est de 100 Mo et **AUTOEXTEND** est défini sans restriction de taille maximale (**MAXSIZE**).
- Il est toutefois possible d'indiquer une taille de fichier.

```
CREATE TABLESPACE tablespace
  [ DATAFILE [ filename ] [ SIZE integer [K|M] ] ];
```

■ Ajouter des fichiers de données aux tablespaces OMF

- Un fichier de données peut être ajouté à un tablespace existant. La commande **ADD DATAFILE** ne requiert plus de spécification de fichier.

Gérer des tablespaces à l'aide d'OMF

• Créer un tablespace OMF

```
CREATE TABLESPACE text_data DATAFILE SIZE 20M;
```

• Ajouter un fichier de données OMF à un tablespace existant

```
ALTER TABLESPACE text_data ADD DATAFILE;
```

• Modifier de manière dynamique l'emplacement du fichier par défaut :

```
ALTER SYSTEM SET
db_create_file_dest = '/u01/oradata/dba01';
```

• Supprimer un tablespace supprime également des fichiers du système d'exploitation.

Gérer des tablespaces à l'aide d'OMF

Gérer des tablespaces à l'aide d'OMF

- **Modifier de manière dynamique les emplacements du fichier par défaut**
 - `DB_CREATE_ONLINE_LOG_DEST_n` doit être défini pour éviter que des fichiers journaux et des fichiers de contrôle ne soient placés avec des fichiers de données. Leur destination peut être modifiée de manière dynamique à l'aide de la commande `SYSTEM SET`.
- **Supprimer les tablespaces OMF**
 - Lorsqu'un tablespace créé avec `OMF` est supprimé, les fichiers de données associés sont **supprimés automatiquement** au niveau du système d'exploitation.

Gérer des tablespaces à l'aide d'OMF

- **Créer un tablespace OMF**

```
CREATE TABLESPACE text_data DATAFILE SIZE 20M;
```

- **Ajouter un fichier de données OMF à un tablespace existant**

```
ALTER TABLESPACE text_data ADD DATAFILE;
```

- **Modifier de manière dynamique l'emplacement du fichier par défaut :**

```
ALTER SYSTEM SET
db_create_file_dest = '/u01/oradata/dba01';
```

- **Supprimer un tablespace supprime également des fichiers du système d'exploitation.**



Annexe 1 : Le cryptage transparent

- Pour pouvoir mettre en place le **cryptage transparent**, il faut d'abord configurer « **Oracle Encryption Wallet** » s'il n'a pas déjà été initialisé pour votre base de données.
- Initialiser consiste à **créer un fichier « WALLET » portefeuille** dans le **répertoire par défaut** pour ce fichier de la base de données : **\$ORACLE_BASE/admin/SID/wallet ou %ORACLE_BASE%\admin\SID\wallet**
- Le répertoire doit exister, sinon la création par défaut du fichier est impossible.
- Vous pouvez visualiser l'emplacement du fichier « **WALLET** » portefeuille et le **statut** en interrogeant la vue « **V\$ENCRYPTION_WALLET** ».
- La syntaxe de **création du portefeuille** est la suivante : `ALTER SYSTEM SET ENCRYPTION KEY IDENTIFIED BY "mot de passe";`

mot de passe : Le mot de passe pour l'activation du cryptage une fois le **fichier portefeuille** créé.

- Vous pouvez visualiser les informations concernant les algorithmes de cryptage en interrogeant la vue « **V\$RMAN_ENCRYPTION_ALGORITHMS** ».

```
SYS@topaze>select algorithm_name "Nom",
2 algorithm_description "Description",is_default "Déf"
3 from v$rman_encryption_algorithms;
```

Nom	Description	Déf
AES128	AES 128-bit key	YES
AES192	AES 192-bit key	NO
AES256	AES 256-bit key	NO

```
SYS@topaze>!mkdir /u01/app/oracle/admin/topaze/wallet
...
drwxr-xr-x. 2 oracle oinstall 4096 8 mars 18:51 wallet <-----
```

Annexe 1 : Le cryptage transparent

```
drwxr-x---. 2 oracle oinstall 4096 7 mars 12:08 xdb_wallet

SYS@topaze>select wrl_type, wrl_parameter, status from v$encryption_wallet;

WRL_TYPE      WRL_PARAMETER                                STATUS
-----
FILE          /u01/app/oracle/admin/topaze/wallet          NOT_AVAILABLE

SYS@topaze>alter system set encryption key identified by "Razvanpwd3";

Système modifié.

SYS@topaze>select wrl_type, wrl_parameter, status from v$encryption_wallet;

WRL_TYPE      WRL_PARAMETER                                STATUS
-----
FILE          /u01/app/oracle/admin/topaze/wallet          OPEN

SYS@topaze>!ls -la /u01/app/oracle/admin/topaze/wallet
...
-rw-r--r--. 1 oracle oinstall 3112  8 mars 18:58 ewallet.p12
```



Il est impératif, après chaque démarrage de la base, d'ouvrir le fichier portefeuille pour pouvoir bénéficier des fonctionnalités de cryptage.

La syntaxe d'ouverture ou de la fermeture du **fichier portefeuille** est la suivante :

```
ALTER SYSTEM SET WALLET {OPEN|CLOSE} IDENTIFIED BY "mot de passe";
```

Annexe 1 : Le cryptage transparent

```

SYS@topaze>startup force
Instance ORACLE lancée.
...
SYS@topaze>select wrl_type, wrl_parameter, status from v$encryption_wallet;

WRL_TYPE          WRL_PARAMETER          STATUS
-----
FILE              /u01/app/oracle/admin/topaze/wallet  CLOSED

SYS@topaze>alter system set wallet open identified by "Razvanpwd3";

Système modifié.

SYS@topaze>create tablespace tbs_ne datafile size 256k;

Tablespace créé.

SYS@topaze>create tablespace tbs_e datafile size 256k
2 encryption using 'AES256' default storage ( encrypt);

Tablespace créé.

SYS@topaze>create table nom01 ( nom varchar2(50)) tablespace tbs_ne;

Table créée.

SYS@topaze>create table nom02 ( nom varchar2(50)) tablespace tbs_e;

Table créée.

```

Annexe 1 : Le cryptage transparent



```

SYS@topaze>insert all into nom01 into nom02 select 'RazvanBIZOI' from dual;

2 ligne(s) créée(s).

SYS@topaze>commit;

Validation effectuée.

SYS@topaze>select * from nom01, nom02

NOM                                NOM
-----
RazvanBIZOI                      RazvanBIZOI

SYS@topaze>alter system set wallet close identified by "Razvanpwd3";

Système modifié.

SYS@topaze>select * from nom02;
select * from nom02
          *
ERREUR à la ligne 1 :
ORA-28365: le portefeuille n'est pas ouvert

```



```

SYS@topaze>select * from nom01;

NOM
-----
RazvanBIZOI

1 ligne sélectionnée.

SYS@topaze>select file_name from dba_data_files join
2 dba_tablespaces using ( tablespace_name)
3 where tablespace_name like 'TBS%E';

FILE_NAME
-----
/u01/app/oracle/oradata/TOPAZE/datafile/o1_mf_tbs_e_9kppzw1w_.dbf
/u01/app/oracle/oradata/TOPAZE/datafile/o1_mf_tbs_ne_9kppz1q1_.dbf

```



Annexe 1 : Le cryptage transparent

```
SYS@topaze>!mv /u01/app/oracle/admin/topaze/wallet/ewallet.p12
/u01/app/oracle/admin/topaze/wallet/ewallet.p12.sav

SYS@topaze>alter system set wallet open identified by "Razvanpwd3";
alter system set wallet open identified by "Razvanpwd3"
*
ERREUR à la ligne 1 :
ORA-28367: le portefeuille n'existe pas

SYS@topaze>alter system set encryption key identified by "Razvanpwd3";
alter system set encryption key identified by "Razvanpwd3"
*
ERREUR à la ligne 1 :
ORA-28362: clé maître introuvable

SYS@topaze>alter system set wallet close identified by "Razvanpwd3";
```



Système modifié.

```
SYS@topaze>!mv /u01/app/oracle/admin/topaze/wallet/ewallet.p12.sav
/u01/app/oracle/admin/topaze/wallet/ewallet.p12
```

```
SYS@topaze>alter system set wallet open identified by "Razvanpwd3";
```

Système modifié.

```
SYS@topaze>!find /u01/app/oracle/oradata/TOPAZE/datafile/o1_mf_tbs_* | xargs grep
'Razvan' -sl
/u01/app/oracle/oradata/TOPAZE/datafile/o1_mf_tbs_ne_9kppz1q1_.dbf
```

```
SYS@topaze>!grep -lR "Razvan" /u01/app/oracle/oradata/TOPAZE/datafile/o1_mf_tbs_*
/u01/app/oracle/oradata/TOPAZE/datafile/o1_mf_tbs_ne_9kppz1q1_.dbf
```



Annexe 1 : Le cryptage transparent

- L'utilisation du fichier portefeuille permet de créer des **tablespaces cryptés**. Ainsi le tablespace **TBS_NE** est un tablespace classique qui contient une table avec un seul champ et un seul enregistrement avec la valeur **'RazvanBIZOI'**. Le deuxième tablespace **TBS_E** est un **tablespace crypté** contenant également une table avec un seul champ et un seul enregistrement.
- Évidemment si vous perdez le **fichier portefeuille** vous n'avez plus **aucune possibilité de lire les données stockées** dans les tablespaces.

```
SYS@topaze>startup force
[oracle@terra ~]$ find /u01/app/oracle/oradata/TOPAZE/datafile/o1_mf_tbs_* | xargs
grep 'Razvan' -sl
/u01/app/oracle/oradata/TOPAZE/datafile/o1_mf_tbs_ne_9kppz1q1_.dbf
[oracle@terra ~]$ grep -lR "Razvan"
/u01/app/oracle/oradata/TOPAZE/datafile/o1_mf_tbs_*
/u01/app/oracle/oradata/TOPAZE/datafile/o1_mf_tbs_ne_9kppz1q1_.dbf
```

- Dans l'environnement Linux/Unix vous pouvez utiliser la commande « **find** » ou « **grep** » pour rechercher la chaîne **'Razvan'** dans le **fichier binaire**. Dans le cas d'un **tablespace crypté** il n'est pas possible de trouver l'information.